# Cavendish

A Gravitational Atom-Gradiometer Simulator

Engineering & Physics Specification

Draft v0.5

July 2, 2026

*A Rust library that simulates the differential phase an atom-interferometer gradiometer array (AION-10 geometry) reads from moving external masses, and emits the full per-measurement state — source kinematics, mass distribution, gravitational field, and the noisy multi-detector signal — as a live stream of tensors. Its purpose is to be the data engine for* Gradar *(passive gradiometric tracking): a learned tracker that recovers the time-resolved tracks of moving masses (gravity-gradient noise) from the joint signal across a spatially separated array, with coherent ultralight dark matter sitting in the same data as the common-mode channel the tracker rejects.*

Malachy

The forward model and validation targets are taken from G. Parish, *Charting New Frontiers in Dark Matter Detection using Atom Interferometry* (KCL/RAL upgrade report, 2026), and his reference implementation, which together serve as the numerical oracle.

# Contents

# 1 Introduction

## 1.1 Purpose

Cavendish simulates the gravitational *gravity-gradient noise* (GGN) signal that moving external masses induce in an array of atom-interferometer gradiometers, and produces the complete physical state of each scenario as machine-learning–ready tensors. Its purpose is to be the data engine for *Gradar*: a learned tracker that solves the inverse problem of recovering the *time-resolved tracks* of the moving sources — their positions over time, hence velocities and trajectories, and how many there are — from the joint multi-detector signal. This is passive (no emission; the masses' own static fields are read as they move), and the spatial separation of the array is what makes position identifiable (Sections 4.2 and 5).

The instrument modelled is a Cavendish-class device. In a laboratory G-measurement the source mass is known and one solves the differential phase for $G$; Cavendish runs the identical forward map inverted — $G$ is held fixed and the source is the unknown. The gradiometer geometry, atom species and timing follow the AION-10 experiment.

## 1.2 Context

The forward model, instrument parameters and validation targets are taken from G. Parish's upgrade report [1] and his reference code, which characterise GGN from oscillating masses for AION-10, extending the constant-velocity treatment of Carlton & McCabe [2]. That work reframes the problem the simulator serves: its scientific target is *ultralight dark matter* (ULDM), a coherent phase oscillation at $f_\phi \approx 0.1\,\mathrm{Hz}$ (Section 4.9); the moving masses (lifts, oscillating plant) are the GGN *background* to be removed. So Cavendish's moving-mass forward model *is* the GGN model, and the two live in the same data split by spatial coherence: a local mass is differential across the array (the Gradar target), a coherent ULDM field is common-mode (the same data, the channel the tracker rejects).

The reference code is adopted as the numerical *oracle*: Cavendish must reproduce it to $\geq 99\%$ on the point-mass cases, converge to its extended-body integrals, and recover the ULDM line through its analysis pipeline (Section 10). The report itself names "machine-learning approaches trained on the simulation outputs developed here" as the next step [1]; Cavendish is that data engine.

## 1.3 Design philosophy

Three principles fix the architecture.

1. **One currency: $\rho(\mathbf{x}, t)$.** The gravity kernel needs only the mass-density field as a function of space and time, discretised to a cloud of mass elements per tick. Every source — a rigid body, a walking person, a sloshing fluid, a vibrating floor — is just a different way of producing $\rho(\mathbf{x}, t)$. The kernel, the forward model and the data contract never change as sources are added.

2. **Live generation.** The library runs as a data source, not a precomputed dataset: a training loop requests scenarios and the library generates and streams them on demand. Disk is an optional cache, not the path.

3. **The engine dumps the entire state; the task lives downstream.** A single typed bundle (Table 8) is the only coupling between the engine and anything downstream, and it is *complete*: the simulator emits the full per-measurement record — source kinematics, mass distribution, field, the *multi-channel* signal (one differential-phase series per detector in the array, Section 4.2), the time-resolved source tracks and their count, the contamination mask,

derived spectra — all as ground truth at generation time. Generating ML datasets at scale is a primary intended use, and the engine is built for it (batched, reproducible from a seed, tensor-native); but it serves that use by emitting the *whole* record and letting the consumer decide what is input, label, target, or context. The supervised, self-supervised, and analysis structure lives entirely downstream — which is precisely what keeps the dump total, and every downstream use (a tracker, a world-model, a Cramér–Rao study, one not yet imagined) possible. A model later produces a prediction against the bundle; a viewer renders either.

# 2 Scope

## 2.1 Version 1 (this specification)

- **Source:** rigid bodies only — arbitrary mesh → voxel cloud, with analytic primitives as the oracle; a library of prescribed/physical trajectories (static, linear pass, 1D/2D/3D oscillation, circular, lift, pendulum, spin, rocking, and free rotation — the Euler top); composition of independent sources, including *multiple simultaneous targets* (the multi-target Gradar case).
- **Instrument:** an *array* of vertical gradiometers at configurable ground placements (Section 4.2); $N$=1 recovers a single gradiometer.
- **Forward model:** the propagation-phase double-difference integral (Section 4.4), evaluated per detector along the four parabolic arm trajectories.
- **ULDM:** the closed-form coherent dark-matter phase (Section 4.9) as an additive common-mode signal, the separation target the Gradar tracker rejects.
- **Gravity:** direct element sum near-field; multipole expansion far-field; swappable discretisation element (point default; sphere, cube).
- **Measurement schedule:** a realistic, seedable cadence model (Section 4.12) — cycle jitter, shot failures, scheduled (e.g. nighttime) gaps, and Poisson transient (lift) events with contamination tagging — yielding non-uniformly sampled series; a uniform default is the degenerate case.
- **Noise:** a pluggable, seedable stack (shot noise, vibration residual); may ship empty and grow, since the forward model validates noiseless.
- **Compute:** the parallel field-evaluation hot path on GPU, the sequential glue on CPU (Section 3.4); CPU-only is a valid fallback for the reference path.
- **Interfaces:** a Python SDK (`run`, `stream`) and thin CLI; a native `egui`/`wgpu` debug viewer.
- **Output:** the state bundle as torch-ready tensors, live — the multi-detector signal, the time-resolved track labels, the Lomb–Scargle periodogram; optional frozen cache.

## 2.2 Deferred (seams kept, not built)

Non-rigid source dynamics (articulated bodies, deforming bodies, compressible and incompressible fluids), structural vibration, airflow/convection (Appendix A); the learned Gradar tracker itself, which *consumes* the stream (Section 5); and the arbitrary-orientation interferometer (v1 arrays are horizontally separated *vertical* gradiometers). The Fisher-information / Cramér–Rao floor on what the array can identify is in scope as an analysis output (Section 5), enabled by the differentiable forward model (NFR 8).

# 3 System Architecture

## 3.1 Crate and module map

Dependency flows one way out of `sim-core`. The SDK is the primary consumer, the viewer secondary: two Rust crates and one Python package.

```
cavendish/
  crates/
    sim-core/             # the engine -- all physics, no Python, no UI
      gravity/            # potential/field/tensor; elements; multipole; oracle primitives
      shape/              # geometry -> mass: primitives + mesh import, voxeliser (Solid
    seam)
      compute/            # CPU reference + GPU (wgpu) field-evaluation backend
      source/             # SourceDynamics (rigid); Trajectory library; bodies via shape
      instrument/         # detector array: gradiometer geometry, placements; arm
    construction; PhaseModel
      uldm/               # closed-form coherent dark-matter phase (common-mode signal)
      noise/              # NoiseSource stack
      scenario/           # Scenario assembly + prior/sampler + measurement Schedule
      state/              # StateBundle (multi-detector signal + ground-truth tracks) +
    field selection
      config/             # the one typed config schema
      generate/           # per-scenario evaluation + batch (GPU dispatch / rayon)
    viewer/               # eframe/egui + wgpu (one-way dep on sim-core)
  sdk/                    # maturin/PyO3 package: run()/stream() + CLI entry
  docs/
```

### 3.2 The seams

Four traits are the flex points; everything else is composition over them. Keeping these clean is what lets the deferred features land as backends without touching the core.

**The four load-bearing traits**

`GravitySource` — produces $V$, $\mathbf{g}$, $\Gamma$ at a world point. Implemented by discretisation *elements*, by the *cloud* (element sum + multipole far-field), and by analytic *oracle* primitives.
`SourceDynamics` — the mass configuration as a continuous function of time, `cloud_at(t)`. `Rigid` specialises it as pose $\otimes$ body-cloud.
`PhaseModel` — turns source + instrument into the differential phase at one measurement time. v1: the propagation integral.
`NoiseSource` — one additive, seedable contribution to the signal; the stack is a `Vec`.

### 3.3 Generation modes

Prescribed sources (Appendix A) run live inside `DataLoader` workers — cheap, stateless per tick, embarrassingly parallel. Solver-driven sources are too slow for training-rate generation and integrate instead via *cache-and-replay*: solve a modest library of $\rho(\mathbf{x}, t)$ fields offline, then place, scale and superpose them live as cheap sources. This is the single point at which the live-generation model bends, and it bends by design.

### 3.4 Compute: the CPU/GPU split

The cut between GPU and CPU is not chosen but read off the dependency structure of the forward map: **the parallel arithmetic and its reductions run on the GPU; the sequential glue, one-time setup and I/O stay on the CPU**. With voxel clouds at $N \sim 10^5$–$10^7$ for extended bodies, the per-measurement field evaluation is the expensive part and is the whole reason for the GPU — a CPU-only path remains valid as the reference but is too slow for training-rate generation.

**GPU — the field-evaluation outer product.** Almost every axis is independent: scenarios, measurements, detectors, interferometers, arms, and the within-flight `fine_dt` integrand samples are mutually independent; the only joins are two *reductions* (the sum over $N$ voxels for $V$

at a point, and the quadrature over the flight for an arm's phase), which a GPU does well. The GPU therefore owns: source-pose and atom-arm evaluation (closed-form, inline); the potential / gradient-tensor sum over the cloud at every field point (the near-field $N$-body reduction); the multipole far-field branch; the flight-integral quadrature; the differential-phase and gradiometer double-difference; and noise injection via a counter-based RNG (philox/threefry), whose draw at each index is parallel rather than sequential.

**CPU — setup, control flow, I/O.** Voxelisation and the multipole reduction run *once per template at load* — structurally parallel but amortised over millions of scenarios, so there is no reason to ship them to the GPU; this is the only compute ever legitimately on the CPU. Scenario sampling (discrete and branchy), kernel launch and batching, and serialisation / the tensor handoff are the rest.

**The one sequential axis, kept on the GPU.** ODE-derived motion (the pendulum and free rotation, NFR 11) is sequential *in time* — which says CPU — but independent *across scenarios*, and the poses it produces are consumed immediately by the field evaluation. The deciding factor is locality, not parallelism: shipping CPU-integrated poses would mean uploading source poses at `fine_dt` resolution (the source moves appreciably *during* the 1.46 s flight), $\sim$ (measurements$\times$ 146) poses per scenario — gigabytes per batch. So the ODE is integrated on the GPU, one thread per scenario, producing poses where they are consumed. The general rule across the bus: **upload parameters, not poses** — tiny trajectory-parameter sets and RNG keys go up, poses are evaluated inline, the fat intermediate never comes back.

**Boundary contract**

**CPU $\rightarrow$ GPU:** the static body cloud, once per template; per-scenario trajectory parameters + RNG keys (tiny); instrument and array placements. **GPU $\rightarrow$ CPU:** the per-detector signal $\Delta\Phi_\ell$ and the per-measurement state for the bundle.

No raw Vulkan: the native side uses `wgpu` compute (it already backs the viewer, sits on Vulkan / Metal / DX12 anyway, and reuses the same device and WGSL shaders); the heavy research-numerics path is vectorised in CUDA via the framework for zero-copy and autodiff. On the `wgpu` path WGSL has no $f64$ and gravity differentials are tiny, so the kernel is written *differential-first* — the gradient tensor / $\Gamma_{zz}L$ evaluated directly, never two large absolute phases subtracted (NFR 9) — keeping $f32$ safely above the mrad shot-noise floor. The multi-detector array adds one dimension to the outer product (scenario $\times$ detector $\times$ timestep $\times$ field point): the same kernel, more field points.

### 3.5 The two representations

The one architectural decision that is genuinely expensive to change later — and therefore the one to fix now — is the boundary between what the gravity kernel consumes and what each source backend evolves. Two voxel representations exist, and they must stay separate.

- **The gravity input** (what the kernel consumes) is the `Cloud`: elements of (position, mass) and at most an element-shape tag, nothing else. This is the universal currency; it stays minimal forever, and the kernel never learns about material, temperature, velocity, pressure or phase.
- **The solver's own state** (what each backend evolves) is where material type, temperature, velocity, pressure and compressibility live — a convection solver carries a temperature/velocity/density grid; a rigid body carries a pose; a vibration solver carries modal amplitudes. Each backend holds whatever rich per-cell state its physics needs and *emits* a minimal `Cloud` through `cloud_at(t)`.

Material phases — `solid` and the compressible / incompressible fluid types — are therefore

the *solver's* cell types, not new fields on the gravity voxel. A single universal voxel carrying {mass, material, temperature, ...} would make the kernel depend on the union of every physical property, so adding any new physics would touch the core — destroying the "$\rho$ is the only currency" decoupling the architecture rests on (Figure 1). Everything behind the cloud — the fluid solver, the temperature field, the skinning — can land years later without touching the core, provided this boundary is kept clean now.

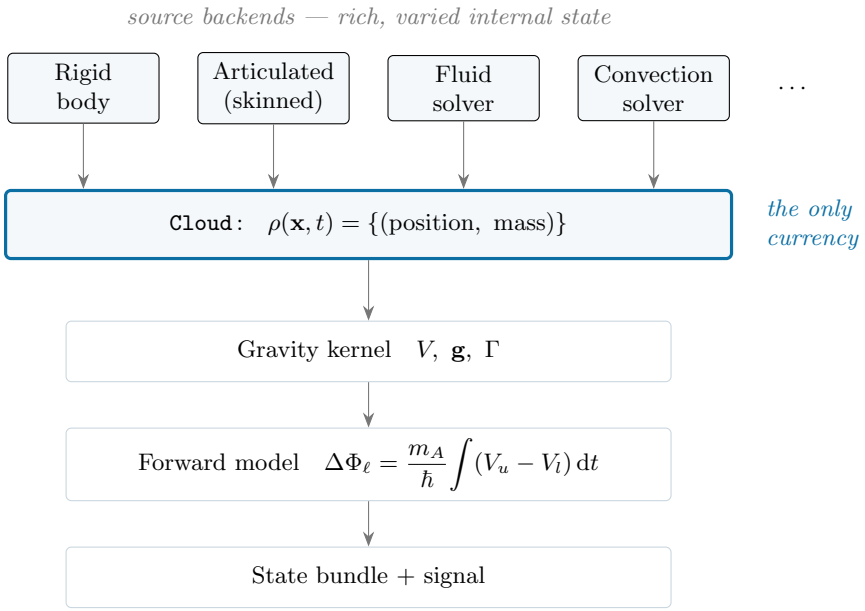


Figure 1: The minimal-cloud chokepoint. Every source backend, whatever its internal state, emits the same minimal $\rho(\mathbf{x}, t)$ cloud; the kernel and forward model see only that. New physics is added as a new backend above the line, never as a change below it.

### 3.6 Conventions

The choices that, left implicit, surface later as a silent mismatch against George's code or the asset importer — fixed here by fiat, which is far cheaper than debugging a wrong frame.

- **Frame and units.** World axes are right-handed with $z$ vertical (the tower axis); gravity acts along $-z$. Lengths in metres, time in seconds, mass in kilograms — SI throughout, with no internal scaling.
- **Orientation.** Quaternions are stored scalar-first $(w, x, y, z)$ in the state bundle; an imported glTF asset (scalar-last, $x, y, z, w$) is converted at the import boundary, the only place the two conventions meet.
- **Units in types.** Physical quantities use newtype wrappers (`Metres(f64)`, `Seconds(f64)`, `Radians(f64)`) so the compiler refuses to add a length to a time — the cheapest defence against the unit bug.
- **Pose ordering.** The per-tick pose is $(\text{position}_3, \text{quaternion}_4)$ and the twist $(\text{linear}_3, \text{angular}_3)$, in that order, matching Table 8.

## 4 Physics and Mathematical Model

This section states the physics Cavendish implements. Equation numbers in parentheses refer to the source report [1]; the model adopted here is the full propagation-phase integral, not the uniform-field shortcut.

### 4.1 The atom gradiometer

The instrument is two Mach–Zehnder atom interferometers stacked vertically and interrogated by common lasers (a gradiometer). Each interferometer splits a cloud of ultracold $^{87}$Sr atoms with a $\frac{\pi}{2}$–$\pi$–$\frac{\pi}{2}$ pulse sequence; the relative phase $\Delta\phi$ accumulated between the two arms sets the output port populations,

$$P_{|g\rangle} = \cos^2\left(\tfrac{1}{2}\Delta\phi\right), \qquad P_{|e\rangle} = \sin^2\left(\tfrac{1}{2}\Delta\phi\right). \tag{1}$$

The two interferometers, launched $\Delta r = 5\,\mathrm{m}$ apart in a 10 m tower, are sensitive to the *difference* in local gravity between them.

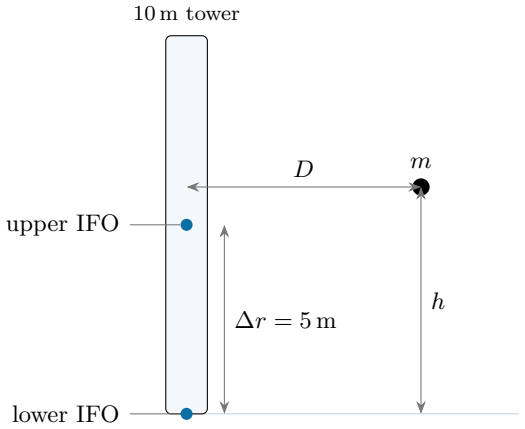


Figure 2: Gradiometer geometry. Two $^{87}$Sr clouds, launched $\Delta r = 5\,\mathrm{m}$ apart in a 10 m tower, interrogate vertically; an external mass at horizontal distance $D$ and height $h$ perturbs the local potential, and the gradiometer reads the *difference* in its pull between the two interferometers.

### 4.2 The gradiometer array and the two channels

A single gradiometer cannot separate a coherent global oscillation from a local gravitational transient at the same frequency; an array of them can, because the two have opposite spatial signatures. The v1 array is several vertical gradiometers (each a tower as above) at distinct ground positions $(x, y)$ — the AION-network geometry; arbitrary orientation is deferred.

**Common-mode vs differential.** A ULDM field (Section 4.9) is spatially coherent over its de Broglie length, which for a $4 \times 10^{-16}$ eV scalar is of order an astronomical unit — vastly larger than any terrestrial array — so every detector reads the *same* phase, in step: it is *common-mode*. A local moving mass is a near-field effect that falls off with distance and looks different metres away: it is *differential*. Hence **common-mode across the array → ULDM, differential across the array → local GGN**. This is the discriminant a single device lacks, and it is what makes the array the substrate for both detecting dark matter and tracking the masses that obscure it.

**Localisation.** The same separation gives source position. Differential amplitude across detectors triangulates distance — and because source mass is common-mode (it scales every detector equally) while position is differential (it changes the ratios), the array *breaks the mass–distance degeneracy* a single gradiometer cannot. Relative timing of closest approach and the differing tensor bearing at each detector add direction. This is gravitational triangulation, the principle of a gravitational-wave or seismic network. Its precision scales as (array baseline / source distance) × SNR, quantified by the Cramér–Rao floor of Section 5; array placement is therefore a design knob, not a fixed constant. Position is recovered from the *joint* multi-detector signal

— forming whatever channel combination is optimal — not by literally differencing detectors, which would reject the common mode and discard the absolute amplitude that carries distance.

### 4.3 Phase decomposition and gradiometer rejection

The end-of-sequence phase is the sum of laser, separation and propagation contributions (58–59),

$$\Delta\phi = \Delta\phi_p + \Delta\phi_\ell + \Delta\phi_s. \tag{2}$$

Because both interferometers share the same lasers, the laser phase $\Delta\phi_\ell$ is common-mode and cancels in the gradiometer difference; the separation phase $\Delta\phi_s$ is negligible. The propagation phase $\Delta\phi_p$ — the part sensitive to the local gravitational potential along the atom trajectories — carries the GGN and is the quantity Cavendish computes.

### 4.4 The forward model

Along the vertical ($z$) flight, an atom follows the semiclassical Lagrangian (62)

$$\mathcal{L} = m_A\big(\tfrac{1}{2}\dot{z}^2 - gz + \tfrac{1}{2}\,\gamma_{zz}z^2 - V(z,t)\big)\,, \tag{3}$$

where $m_A$ is the atomic mass, $g$ the local gravitational acceleration, $\gamma_{zz}$ the static Earth gradient, and $V(z,t)$ the perturbing potential of the external source. For a point source (63),

$$V(z,t) = -\frac{G\,m}{\sqrt{D(t)^2 + \big(z(t) - h(t)\big)^2}}, \tag{4}$$

and for a general mass distribution, discretised to a cloud of $N$ elements of mass $m_i$ at world positions $\mathbf{r}_i(t)$,

$$\boxed{V(\mathbf{r},t) = -G\sum_{i=1}^{N}\frac{m_i}{\|\mathbf{r} - \mathbf{r}_i(t)\|}} \tag{5}$$

which is the potential Cavendish evaluates. The phase imprinted on a single interferometer at measurement $\ell$ is the time integral of the potential *difference between its two arms* over the sequence (64),

$$\boxed{\delta\phi_\ell = \frac{m_A}{\hbar}\int_{\ell\,\Delta t}^{\ell\,\Delta t + 2T}\big[\,V(z_u,t) - V(z_l,t)\,\big]\,\mathrm{d}t} \tag{6}$$

where $z_u, z_l$ are the upper and lower arm trajectories. The gradiometer observable is the *double difference* across the two interferometers (65),

$$\boxed{\Delta\Phi_\ell = \delta\phi_{\ell,2} - \delta\phi_{\ell,1}} \tag{7}$$

($\delta\phi_{\ell,2}$ upper, $\delta\phi_{\ell,1}$ lower; the overall sign is convention). The signal is the series $\{\Delta\Phi_\ell\}$ over measurements $\ell = 0, \ldots, N-1$. Note $\Delta\Phi \propto m$: the response is linear in source mass.

### 4.5 Trajectory construction and the recoil scale

The four arm trajectories depend only on the instrument and are built once. Each interferometer has an upper and a lower arm; large-momentum-transfer (LMT) kicks impart a recoil velocity

$$v_{\mathrm{rec}} = n_{\mathrm{kick}}\,\frac{\hbar k}{m_A}, \qquad k = \frac{2\pi}{\lambda}, \tag{8}$$

with $n_{\mathrm{kick}} = 1000$ photon kicks at $\lambda = 698\,\mathrm{nm}$, giving $v_{\mathrm{rec}} \approx 6.5\,\mathrm{m/s}$ — the arm-separation velocity that sets the *absolute* signal scale. The arms are ballistic free-fall under $-g$ from the launch velocity, with a reflecting floor at the launch height:

- **Upper arm:** launch with $+v_{\text{rec}}$ on $[0, T]$; apply $-v_{\text{rec}}$ at $t = T$ (the $\pi$ pulse) on $[T, 2T]$.
- **Lower arm:** launch at the cloud velocity on $[0, T]$; apply $+v_{\text{rec}}$ at $t = T$ on $[T, 2T]$.

The two interferometers launch at $z_0 = 0$ and $z_0 = \Delta r$.

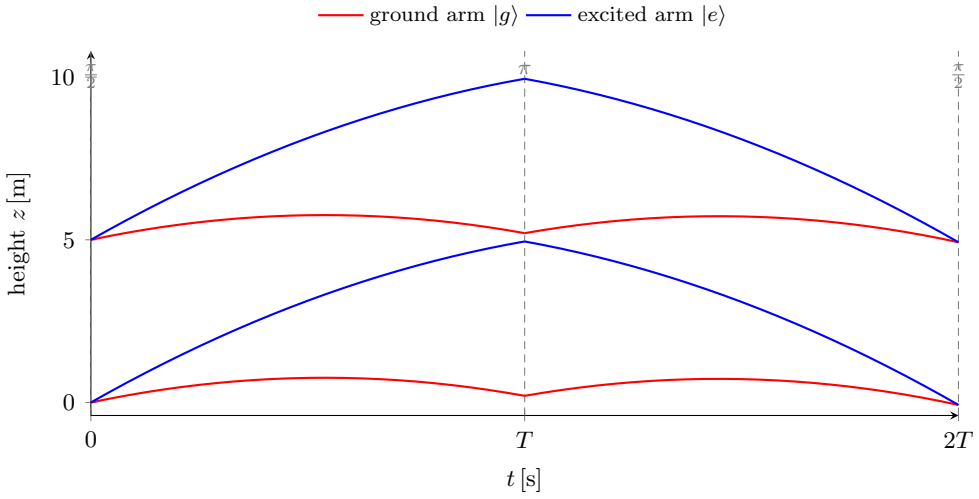


Figure 3: Atom trajectories in the gradiometer, computed from the forward model with the AION-10 parameters (Table 2). Within each interferometer the ground ($|g\rangle$) and excited ($|e\rangle$) arms separate at the first $\frac{\pi}{2}$ pulse, swap momentum at the $\pi$ pulse and recombine at $2T$; the two interferometers are offset by $\Delta r = 5\,\text{m}$. The propagation phase is the source potential integrated along these paths.

## 4.6 The mass model

### 4.6.1 Discretisation elements

A voxel is the elemental mass unit summed to obtain a body's field; the element is a strategy behind `GravitySource`, orthogonal to dynamics and to the far-field path.

- **Point** (v1 default): $V = -Gm/\|\mathbf{r}-\mathbf{r}_0\|$ — cheapest, far-field-justified, singular at the element.
- **Softened sphere:** $V = -Gm/\sqrt{\|\mathbf{r}-\mathbf{r}_0\|^2 + \varepsilon^2}$ — externally near-identical to a point, regularised close in.
- **Uniform cube** (Nagy prism [7]): the exact closed-form potential of a homogeneous rectangular prism — the physically exact voxel, best near the body, costlier per element.

### 4.6.2 Field and gradient tensor

The acceleration and gravity-gradient tensor are

$$\mathbf{g} = -\nabla\Phi, \qquad \Gamma_{ij} = \frac{\partial g_i}{\partial x_j} = -\frac{\partial^2\Phi}{\partial x_i\,\partial x_j}, \tag{9}$$

with $\Phi \equiv V$ the potential per unit mass. By Poisson's equation $\nabla^2\Phi = 4\pi G\rho$, so $\Gamma$ is symmetric and, in vacuum, trace-free ($\operatorname{tr}\Gamma = -4\pi G\rho = 0$) — a free correctness invariant (Section 10).

### 4.6.3 Multipole expansion (far field)

Beyond a cutoff distance a rigid body's field is evaluated from its multipole moments, computed once per body and re-expressed from the pose each tick:

$$\Phi(\mathbf{r}) = -G\left[\frac{M}{r} + \frac{\mathbf{p}\cdot\hat{\mathbf{r}}}{r^2} + \frac{1}{2}\frac{\hat{\mathbf{r}}^{\top}\mathsf{Q}\,\hat{\mathbf{r}}}{r^3} + \cdots\right], \tag{10}$$

with monopole $M = \sum_i m_i$, dipole $\mathbf{p} = \sum_i m_i(\mathbf{r}_i - \mathbf{r}_{\mathrm{com}})$ (vanishing about the centre of mass), and quadrupole moment $\mathsf{Q}$. The quadrupole and the body's *inertia* tensor are linear maps of the same second moment $\mathsf{C} = \sum_i m_i(\mathbf{r}_i - \mathbf{r}_{\mathrm{com}})(\mathbf{r}_i - \mathbf{r}_{\mathrm{com}})^{\top}$ — $\mathsf{Q} = 3\mathsf{C} - (\operatorname{tr}\mathsf{C})\mathbb{1}$ and $\mathsf{I} = (\operatorname{tr}\mathsf{C})\mathbb{1} - \mathsf{C}$ — so they share principal axes, and the same load-time reduction that yields $\mathsf{Q}$ also yields the moments that drive free rotation (Section 7.3). Near the body the field reverts to the direct element sum of Equation (5); the element type matters only there. The report confirms empirically that sources beyond a few metres are well approximated as point masses, validating this cutoff.

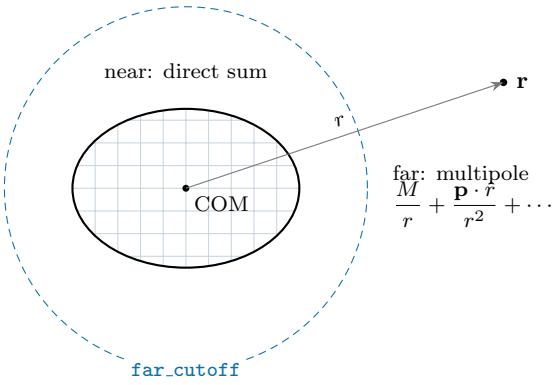


Figure 4: Voxelisation and the near/far split. A body is filled with mass elements (the voxel cloud). At a field point $\mathbf{r}$ within `far_cutoff` the potential is the direct element sum (Equation (5)); beyond it, the body's multipole moments suffice. The discretisation element matters only in the near region.

### 4.7 Analytic primitives (the oracle)

Closed-form / quadrature potentials of homogeneous primitives are the reference the voxel cloud is validated against; they are *not* used in generation. For a cylinder of mass $m$, radius $R$, height $H$ (122),

$$V(\mathbf{r}) = -\frac{Gm}{\pi R^2 H}\int_{-H/2}^{H/2}\int_0^R\int_0^{2\pi} \frac{r'\,\mathrm{d}\theta\,\mathrm{d}r'\,\mathrm{d}h'}{\sqrt{(D - D_0 - r'\cos\theta)^2 + (w - w_0 - r'\sin\theta)^2 + (h - h_0 - h')^2}}, \tag{11}$$

and for a cuboid of side lengths $a, b, c$ (121),

$$V(\mathbf{r}) = -\frac{Gm}{abc}\iiint_{\mathrm{box}} \frac{\mathrm{d}w'\,\mathrm{d}h'\,\mathrm{d}D'}{\sqrt{(D - D')^2 + w'^2 + (h - h')^2}}, \tag{12}$$

with the planar sheet (118/119) the $c \to 0$ surface analogue. A voxelised primitive must converge to these as `voxel_size` $\to 0$ — the proof that the voxel approach is correct.

### 4.8 Quasi-static limit (sanity check only)

For a uniform field the single-interferometer response reduces to the closed form (51)

$$\Delta\phi \approx k_{\mathrm{eff}}\, a\, T^2, \tag{13}$$

retained only as a uniform-field cross-check. GGN sources are non-uniform, so Equation (6) is the model.

### 4.9 The ULDM signal (common-mode channel)

The scientific target is a coherent, closed-form phase — not a gravitational source at all, but an oscillation of the atomic transition frequency driven by a scalar dark-matter field. It is additive to the GGN signal and identical across the array (Section 4.2). For a gradiometer of baseline $L$ and separation $\Delta r$ with $n$ LMT kicks [10],

$$\Delta\Phi_\phi(t) = 8\,\frac{\delta\omega_A}{\omega_\phi}\,\frac{\Delta r}{L}\,\sin\!\frac{\omega_\phi n L}{2c}\,\sin\!\frac{\omega_\phi\big(T-(n-1)L/c\big)}{2}\,\sin\!\frac{\omega_\phi T}{2}\,\cos\!\Big(\omega_\phi\big(\tfrac{2T+L/c}{2}+t\big)+\theta\Big), \quad (14)$$

with $\omega_\phi = 2\pi f_\phi$ and the transition-frequency modulation amplitude

$$\delta\omega_A = \omega_A\,\sqrt{4\pi G_\mathrm{N}}\,\big(d_{m_e} + (2+\xi)\,d_e\big)\,\frac{\sqrt{2\rho_\phi}}{m_\phi}\,\kappa, \quad (15)$$

where $\omega_A$ is the atomic transition frequency, $d_e, d_{m_e}$ the dilatonic couplings, $\xi$ the clock-transition coefficient, $\rho_\phi$ the local dark-matter density, $m_\phi$ the scalar mass, $G_\mathrm{N}$ Newton's constant in natural units, and $\kappa$ a unit conversion (Table 4). The oscillation frequency is set by the particle mass: $f_\phi = m_\phi c^2/h$ (and $4\times10^{-16}\,\mathrm{eV} \to 0.097\,\mathrm{Hz}$), so *recovering the line is measuring the dark-matter mass.* Because Equation (14) is cheap and deterministic, it is evaluated directly rather than through the gravity kernel; in a scenario it is the common-mode channel the Gradar tracker (Section 5) learns to reject and the line the analysis pipeline (Section 4.10) recovers.

### 4.10 Frequency domain

The signal is naturally analysed as a periodogram. On a *uniform* grid this is the discrete spectrum (66),

$$S_k = \frac{\Delta t}{N}\left|\sum_{\ell=0}^{N-1}\Delta\Phi_\ell\,e^{-2\pi i\ell k/N}\right|^2, \qquad f_k = \frac{k}{N\Delta t}, \quad (16)$$

with Nyquist frequency $f_\mathrm{N} = 1/(2\Delta t)$ and resolution $\Delta f = 1/(N\Delta t)$. Oscillating sources produce peaks at integer multiples $nf_0$ (from the nonlinearity of $V$), with mass-independent peak-height ratios; harmonics above $f_\mathrm{N}$ mirror back.

**The realistic schedule is non-uniform, so the FFT does not apply.** Cycle jitter, shot failures and scheduled gaps (Section 4.12) leave the measurement times $\{t_\ell\}$ irregular, on which the FFT — which assumes even spacing — produces spurious structure. The correct estimator is the **Lomb–Scargle periodogram**, which fits sinusoids by least squares at each trial frequency and handles arbitrary sampling:

$$S(f) = \frac{1}{2}\left[\frac{\big(\sum_\ell y_\ell\cos\omega(t_\ell-\tau)\big)^2}{\sum_\ell\cos^2\omega(t_\ell-\tau)} + \frac{\big(\sum_\ell y_\ell\sin\omega(t_\ell-\tau)\big)^2}{\sum_\ell\sin^2\omega(t_\ell-\tau)}\right], \quad (17)$$

$\omega = 2\pi f$, with the offset $\tau$ fixing time-shift invariance [9]. The Lomb–Scargle transform is the first-class spectral output; the uniform FFT is retained as the fast path for the degenerate evenly-sampled case. GGN removal before reading the ULDM line is a *least-squares notch* at each oscillator frequency *and its second harmonic* (the harmonic because a large-amplitude oscillation is anharmonic, Section 4.10 and the pendulum of Section 7.3); a robust detection floor is the median of $\sqrt{S}$ off the peak.

### 4.11 The two time scales

The within-flight integration step `fine_dt` $\approx 0.01\,\mathrm{s}$ evaluates each $\delta\phi$ over the flight $2T = 1.46\,\mathrm{s}$; the measurement cadence $\Delta t = 2\,\mathrm{s}$ is the spacing at which $\Delta\Phi_\ell$ is sampled (it sets $f_\mathrm{N}$ and the spectral mirroring). These are distinct axes and must not be conflated.

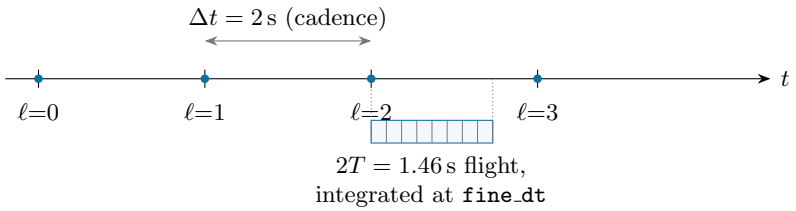


Figure 5: The two time scales. Measurements are spaced by the cadence $\Delta t = 2\,\mathrm{s}$ (which sets $f_\mathrm{N} = 1/2\Delta t$); each $\Delta\Phi_\ell$ is itself a phase integrated over a $2T = 1.46\,\mathrm{s}$ atom flight at the fine step `fine_dt`. The flight is shorter than the cadence — the remainder is dead time.

### 4.12 Measurement schedule and contamination

A real run is not an unbroken uniform series; the measurement times carry structure that the analysis (Section 4.10) and the Gradar front-end (Section 5) must contend with, so the schedule is modelled explicitly and the bundle carries the actual $\{t_\ell\}$ rather than assuming a uniform $\Delta t$. Four effects, each seedable, compose:

- **Cycle jitter** — the cycle duration varies (e.g. $T_c \sim \mathcal{U}(1.9, 3.0)\,\mathrm{s}$, or a per-shot $\pm$ offset), so $\{t_\ell\}$ is non-uniform.
- **Shot failures** — each measurement is dropped independently with probability $p_\mathrm{fail}$ (MOT/preparation failure).
- **Scheduled gaps** — the device runs only on a duty schedule (e.g. nighttime), with jitter on the switch times, leaving daily gaps over multi-day runs.
- **Transient events** — large near-field disturbances (a lift: a heavy mass on a clipped vertical pass) arrive as a Poisson process and contaminate the cycles they overlap; these are *tagged* so they can be excised (with padding) or handed to the model as a contamination mask.

The uniform, gap-free, failure-free series is the degenerate default. The lift is just a `Lift` motion (Section 7.3) flagged as a transient; excision is a mask over the cadence axis, not a physics change.

### 4.13 Noise model

The clean signal is corrupted by an ordered, seedable stack:

$$\sigma_\phi \approx \frac{1}{C\sqrt{N_\mathrm{atoms}}} \quad \text{(atom shot noise)}, \tag{18}$$

with $C$ the fringe contrast, plus a vibration residual with a gradiometer common-mode rejection ratio. Noise is additive realism layered after the forward model is validated.

### 4.14 Atmospheric gravity-gradient noise

Beyond discrete moving masses, the dominant low-frequency GGN in a vertical interferometer is *distributed and stochastic*: fluctuations in the density of the air above the instrument perturb the local potential. Carlton *et al.* [3] give the first characterisation for vertical AIs, in exactly Cavendish's 0.01 Hz to 1 Hz band — the ULDM line at 0.1 Hz sits inside it. The key structural point: it enters through the *same* potential integral as a rigid body (Section 4.4 and fig. 4); only the source is a realised random density field $\delta\rho$ rather than a voxel cloud. So atmospheric GGN is "a source whose mass distribution is a stochastic field," and the existing gravity kernel and forward model evaluate it unchanged — it is the principal growth target for the otherwise placeholder noise stack.

Two channels, each mapping a fluctuation to a density perturbation $\delta\rho/\rho_0$ and thence to a vertical acceleration on the arms:

- **Infrasound (pressure).** Adiabatic, $\delta\rho/\rho_0 = \delta p/(\gamma p_0)$, modelled as plane waves reflecting off the half-space surface at incidence $\theta$; this gives a *closed-form* vertical acceleration and per-shot phase (cheap, like the ULDM signal). Amplitudes come from the global high/low ambient-infrasound models (Bowman; Kristoffersen), with a typical $\delta p \sim 10^{-3}$ mbar and strong (up to two-decade) seasonal variation.
- **Temperature (turbulence).** $\delta\rho/\rho_0 = \delta T/T_0$, from thermal eddies in the surface boundary layer advected past the tower at wind speed $U$ (Taylor's frozen-turbulence hypothesis, $k_x = \omega/U$). The eddy wavenumber spectrum takes the Greenwood–Tarazano form $\Phi(k) \propto (k^2 + k/\Lambda)^{-11/6}$ (Kolmogorov $k^{-11/3}$ in the inertial subrange, corrected at low frequency), outer scale $\Lambda$ at the Obukhov length; the resulting PSD is numerical, with no closed form.

**Gradiometer and array structure.** Unlike a horizontal LIGO-like pair, the two clouds of a *vertical* gradiometer feel *correlated* atmospheric noise along the baseline, so the double-difference partially rejects it — the noise is the subtracted spectrum $S_g(f, z_0) - S_g(f, z_0 + \Delta z)$, and the cloud nearest the surface dominates. Crucially for the array (Section 4.2), atmospheric GGN has a finite *spatial correlation length* — the infrasound wavelength ($\approx 343$ m at 1 Hz) and the eddy/outer scale ($\sim 170$ m) — so across horizontally separated detectors it is only *partially* correlated. This places it between the two limits the array already exploits: fully common-mode (ULDM, coherent over astronomical scales) and fully local (a nearby mass). The array's spatial sampling is therefore what separates all three regimes, and this distributed background is the realistic floor the Gradar front-end (Section 5) must detect through. **Depth** is the passive mitigation: temperature noise falls off sharply with $z_0$, while vertically incident infrasound penetrates furthest.

The transfer from acceleration to gradiometer phase is the sequence's own response $T_\phi = 2n\kappa L \cos(\omega T)\sin^2(\omega T/2)$ (LMT order $n$, laser wavevector $\kappa = \omega_a/c$), which Cavendish's forward model already embodies; its zeros notch the noise at interrogation-time–dependent frequencies. Table 1 lists the parameters.

Table 1: Atmospheric GGN parameters [3].

| Symbol | Quantity | Value |
|---|---|---|
| $\rho_0$ | mean air density | $1.3\,\mathrm{kg/m^3}$ |
| $p_0$ | mean pressure | 1013 mbar |
| $\gamma$ | adiabatic index (air) | 1.4 |
| $T_0$ | mean air temperature | 300 K |
| $c_T$ | temperature structure constant | $0.2\ \mathrm{K^2\,m^{-2/3}}$ |
| $\Lambda$ | turbulence outer scale (Obukhov) | $\sim 170$ m |
| $U$ | mean wind speed | $\sim 20$ m/s |
| $\delta p$ | typical infrasound amplitude | $\sim 10^{-3}$ mbar |
| band | relevant frequencies | 0.01 Hz to 1 Hz |

### 4.15 Reference parameters

Table 2 lists the AION-10 defaults; Table 3 the published regression targets; Table 4 the dark-matter signal parameters. The clock isotope is $^{87}$Sr (the 698 nm clock transition fixes this). The mass $m_A = 1.46 \times 10^{-25}$ is a fixed calibration constant supplied by collaborators — the natural-abundance (isotopic-mixture) value, $\approx 87.6$ u, not $^{87}$Sr's single-isotope $1.443 \times 10^{-25}$. Cavendish keeps this value to reproduce the oracle; the forward model is linear in $m_A$, so the $\sim$1% difference is immaterial.

Table 2: AION-10 instrument defaults (from [1] and reference code).

| Symbol | Quantity | Value | |
|---|---|---|---|
| $G$ | gravitational constant | $6.674\,30 \times 10^{-11}$ | $\mathrm{m^3/(kg\,s^2)}$ |
| $m_A$ | atomic mass ($^{87}$Sr) | $1.46 \times 10^{-25}$ | kg |
| $\hbar$ | reduced Planck constant | $1.055 \times 10^{-34}$ | J s |
| $\lambda$ | clock wavelength | $698 \times 10^{-9}$ | m |
| $k$ | wavenumber $2\pi/\lambda$ | $\approx 9.0 \times 10^{6}$ | 1/m |
| $T$ | pulse separation | 0.73 | s |
| $2T$ | flight time | 1.46 | s |
| $n_{\mathrm{kick}}$ | photon kicks (LMT) | 1000 | — |
| $v_{\mathrm{rec}}$ | recoil/arm velocity | $\approx 6.5$ | m/s |
| $g$ | local gravity | 9.81 | $\mathrm{m/s^2}$ |
| $\Delta r$ | baseline (IFO separation) | 5 | m |
| | tower height | 10 | m |
| $u_0$ | launch velocity | 3.86 | m/s |
| $\Delta t$ | measurement cadence | 2 | s |
| `fine_dt` | integration step | 0.01 | s |

Table 3: Published GGN regression targets (numerical oracle).

| Source | Scenario | Peak $\lvert\Delta\Phi\rvert$ |
|---|---|---|
| Point mass | 10 kg, $D = 10$ m, vertical pass 2.5 m/s | $\approx 7$ mrad |
| Point oscillator | 1 kg, $D_0 = 5$ m, $h = 1 + \cos(2\pi \cdot 0.1\,t)$ | $\approx 2$ mrad |
| Concrete wall (cuboid) | $0.225 \times 6.1 \times 12.2$ m, 0.1 Hz, 1 µm | $\approx 50$ µrad |
| AION scaffold (cylinders) | multiple members, 0.2 Hz, 1 µm | $\approx 10$ µrad |

# 5 Gradar: the Inference Target

Cavendish is the data engine for *Gradar*: a learned tracker that recovers the time-resolved tracks of moving masses from the array signal. The tracker itself is downstream (deferred), but its target is what fixes the label, the prior and the analysis outputs, so it is specified here.

## 5.1 Tracking, not static localisation

The task is tracking, not a single fix: a mass moves through the array's field of view and its track — position over time, hence velocity and trajectory — is reconstructed from how the gradient signature sweeps across the detectors. **The motion is the information**: a stationary mass gives one geometry, a moving one sweeps through many, and that temporal diversity is what makes the inverse well-posed (the same reason a single-axis gradiometer can recover a trajectory at all). It is *passive* — nothing is emitted; the masses' own static fields are read as they move, so there is no time-of-flight ranging, and range comes from amplitude and the cross-detector triangulation of Section 4.2. The array is a phased gravimetric aperture; the tracker is the inference over it.

## 5.2 Why a JEPA

The target maps onto a temporal joint-embedding predictive architecture: past gradient-signature windows predict the latent of future windows, so the latent must encode the sources' kinematic state because that state generates the next window. A world-model latent is literally "predict where it will be" — tracking. The ladder is supervised single-target tracker → temporal JEPA (masked-future prediction, a probe reading off the track) → world-model tracker (online). The multi-detector signal is the natural multi-channel input, and the cross-channel structure

Table 4: Dark-matter (ULDM) signal parameters for Equation (14) (from the reference code).

| Symbol | Quantity | Value | |
|---|---|---|---|
| $m_\phi$ | scalar (ULDM) mass | $4 \times 10^{-16}$ | eV |
| $f_\phi$ | oscillation freq. $m_\phi c^2/h$ | $\approx 0.1$ | Hz |
| $\rho_\phi$ | local dark-matter density | 0.3 | GeV/cm$^3$ |
| $\omega_A$ | atomic transition frequency | $2.67 \times 10^{15}$ | rad/s |
| $d_e,\ d_{m_e}$ | dilatonic couplings | 1 | — |
| $\xi$ | clock-transition coefficient | 0.06 | — |
| $\theta$ | initial phase | $\pi/2$ | rad |
| $G_\mathrm{N}$ | Newton constant (natural units) | $6.708 \times 10^{-39}$ | 1/GeV$^2$ |
| $\kappa$ | unit conversion (nat. $\to$ SI) | $2.77 \times 10^{-12}$ | — |

— common-mode for ULDM, differential for GGN, ratios and timings for position — is exactly what the representation should capture.

### 5.3 Where the difficulty (and the experiment) lives

- **Multi-target data association.** Several masses superpose *linearly* in the field — trivial to simulate (add the clouds) — but their tracks must be disentangled from the summed signal. Linearity helps the forward model and nothing in the inverse; the unmixing is the hard part, the classical radar-tracker (Kalman/JPDA) problem where a learned tracker can win.
- **Degeneracy broken by motion.** A heavy/slow/far mass and a light/fast/near one can *momentarily* alias, but their tracks diverge, so tracking over time resolves what a snapshot cannot. The dedicated confusion-pair prior (Section 7) — instantaneously degenerate pairs — is the clean test of whether the world-model actually uses time.
- **Detection before tracking.** Whether a target is present at all, under the GGN and the schedule gaps, makes the measurement schedule (Section 4.12) the Gradar front-end rather than an afterthought.

### 5.4 What the signal recovers: the identifiability ladder

The exterior field is a multipole series, and recoverability falls off with moment order, so the expansion already in the model (Figure 4) *is* the identifiability ladder: "what can be recovered" is answered moment by moment, not all at once.

**Trajectory and total mass (monopole).** The centre-of-mass motion lives entirely in the monopole — the strongest, slowest-falling term — so position over time, hence velocity, is the robust content; total mass is the monopole strength, recoverable but mass–distance degenerate for a single device and disambiguated by the array (Section 4.2).

**Shape, and exactly how hard (quadrupole).** The dipole vanishes about the centre of mass, so shape onsets at the quadrupole, and the quadrupole/monopole signal ratio scales as $(a/r)^2$ in source size $a$ and distance $r$ — invariant whether $V$, $\mathbf{g}$ or $\Gamma$ is measured, since the extra $1/r$ powers cancel in the ratio. That single number is the entire shape-SNR budget: order-unity for the scaffold (members and distances both metre-scale), negligible for a distant lift. The T2.4 departure-from-monopole sweep (Table 9) is the forward map of this budget.

**Interior density is unrecoverable in principle.** The exterior inverse problem is non-unique: a uniform shell is identical to a point mass, and interior mass can be redistributed across an equivalence class without changing any external moment. Free-form (voxel) density reconstruction is therefore *impossible*, not merely ill-conditioned — no SNR or baseline recovers what the field does not carry.

**Shape as model selection.** Restricting to a shape family is the regulariser that makes the inverse well-posed: it selects one representative of the equivalence class. The target is thus *model selection* over a dictionary (sphere, cylinder, cuboid, sheet) with pose and scale, not field inversion — finite-dimensional, well-posed, and asking only for the low moments that are recoverable at all. The dictionary is the engine's own parameterised body library, so the inference target and the generative prior are the same object, and the shape label is a library entry plus parameters rather than a new representation; for the JEPA this is a shape-class posterior with continuous moment regression.

**The moments close the geometry.** The shape prior does more than classify. For a uniform cylinder the quadrupole fixes only the elongation combination $\mathsf{Q}_{zz} \propto H^2 - 3R^2$; adding the class and a uniform-density assumption closes the system — the monopole gives $M = \rho\pi R^2 H$ and the quadrupole gives $H^2 - 3R^2$, two equations for $(R, H)$. The aspect ratio $H = \sqrt{3}\,R$ gives $\mathsf{Q}_{zz} = 0$, a cylinder indistinguishable from a sphere at quadrupole order — a genuine blind spot. A single vertical gradiometer reads one projection of $\Gamma$, hence of $\mathsf{Q}$, giving elongation magnitude only; the array samples $\mathsf{Q}$ from several bearings, reconstructing the full tensor whose eigenvectors are the principal axes — so orientation becomes observable, "it is elongated" sharpening to "it is a cylinder pointing that way." None of this repeals $(a/r)^2$.

**Rotation isolates the quadrupole.** A source rotating about a fixed centre of mass (Section 7.3) holds the monopole fixed and the dipole zero, so its time-varying signal is *purely* quadrupole-and-higher — the exact complement of monopole-dominated translation, and the cleanest available probe of this rung. A freely tumbling asymmetric body goes further: since its inertia and quadrupole tensors share principal axes, the flip dynamics and the static shape signature are two views of one tensor, and the body over-constrains its own geometry.

**Velocity** is the derivative of the recovered track, so its limits are temporal: the within-flight smear (each $\Delta\Phi_\ell$ is a 1.46 s integral, so motion *during* a flight blurs) and the 2 s cadence — ample for 0.1 Hz GGN, a wall for anything fast.

Table 5: The recoverability ladder: what the signal carries, by multipole order.

| Content | Source | Status |
|---|---|---|
| Position over time, velocity | monopole (CoM motion) | robust, array-aided |
| Total mass | monopole strength | recoverable; mass–distance degenerate, broken by the array |
| Elongation magnitude | quadrupole, one projection | marginal — the $(a/r)^2$ budget |
| Orientation, shape moments | full $\mathsf{Q}$ tensor | array only; still $(a/r)^2$-limited |
| Geometry $(R, H)$, known class | monopole + quadrupole + $\rho$ | closes *given* the shape prior |
| Interior density $\rho(\mathbf{x})$ | — | impossible in principle |

## 5.5 Two channels, one array; and the identifiability floor

Gradar and ULDM detection are not competing framings but the differential and common-mode halves of the same multi-channel problem (Section 4.2): the tracker works the differential structure and rejects the common-mode ULDM; the dark-matter search works the common mode and rejects the differential GGN. The rigorous ceiling on both is the Fisher-information / Cramér–Rao bound, computed from the differentiable forward model (NFR 8): the track (position and velocity) covariance given the array geometry and SNR; the quadrupole detectability that sets whether shape is accessible at all (the $(a/r)^2$ budget of Section 5.4) and the model-selection power between shape classes; and a multi-target resolvability measure. A tracker is then judged against this floor, and array placement is optimised against it.

# 6 Requirements

## 6.1 Functional requirements

**FR-1** (*Mesh import and voxelisation*). The system shall construct source mass clouds from analytic primitives (sphere, shell, cuboid, cylinder, and unions thereof, each with closed-form moments as a validation oracle) and from imported triangle meshes (explicit scale; no unit guessing) discretised at a configurable resolution `voxel_size`, with a configurable element kind (point, softened sphere, cube). Every produced cloud shall have exact total mass (renormalised), its centre of mass at the body-frame origin (zero dipole), and a deterministic element order. Open or non-manifold meshes shall be classified robustly (generalised winding numbers) with a diagnostic, failing loudly when the interior is ambiguous.

**FR-2** (*Gravity evaluation*). The system shall evaluate the gravitational potential $V$, acceleration $\mathbf{g}$ and gradient tensor $\Gamma$ at arbitrary world points for single elements, for a voxel cloud, and for analytic primitives, routing near-field points to a direct element sum and far-field points to a multipole expansion.

**FR-3** (*Analytic oracle primitives*). The system shall provide homogeneous sphere, cuboid, cylinder and sheet primitives with closed-form or quadrature potentials, for validating the voxel cloud (Section 10). These shall not be used in signal generation.

**FR-4** (*Rigid source dynamics*). The system shall represent a rigid body as a body-frame cloud plus a continuous pose trajectory `pose_at`$(t)$, built from a path, a timing law, an orientation law and a rigid placement (Section 7.3), with named motions covering at minimum static, linear pass, oscillation, circular, lift, pendulum, and rotation (spin, rocking, and the free-rotation Euler top), composable by superposition and by time-sequencing.

**FR-5** (*Composite and multi-target sources*). The system shall represent a scene as a superposition of independent sources, summing their clouds (gravity is linear). This shall support the multi-cylinder scaffold case and *multiple simultaneous moving targets*, each with its own trajectory, as the multi-target Gradar scenario.

**FR-6** (*Instrument and arm construction*). The system shall model the AION-10 gradiometer and construct the four parabolic arm trajectories with LMT recoil (Section 4.5), caching them as interpolable paths over $[0, 2T]$, with all parameters of Table 2 configurable.

**FR-7** (*Detector array*). The system shall model an array of gradiometers at configurable ground placements (Section 4.2), evaluating the forward model independently per detector and emitting one differential-phase channel per detector, such that $N{=}1$ reproduces a single gradiometer and the placement geometry is a free parameter (and an analysis knob, Section 5).

**FR-8** (*ULDM signal*). The system shall provide the closed-form coherent dark-matter phase (Equation (14)) as an additive, common-mode signal applied identically across the array, with the parameters of Table 4 configurable, and shall be able to enable or disable it per scenario.

**FR-9** (*Forward model*). The system shall compute the differential phase $\Delta\Phi_\ell$ via the propagation-phase double-difference integral (Equations (6) and (7)), querying the source position at `fine_dt` resolution within each flight. A quasi-static gradient model shall be available as an alternative for uniform-field cross-checks.

**FR-10** (*Sampling and Nyquist*). The system shall sample the signal at the measurement cadence `cycle_time`, and expose enough metadata that periodograms and Nyquist mirroring are computed correctly.

**FR-11** (*Measurement schedule*). The system shall optionally apply a seedable measurement schedule (Section 4.12) — cycle jitter, independent shot failures, scheduled (duty-cycle) gaps, and Poisson transient events — producing a non-uniform set of measurement times $\{t_\ell\}$ carried explicitly in the bundle, with transient-contaminated cycles tagged for excision or masking. A uniform, gap-free schedule shall be the default.

**FR-12** (*Noise stack*). The system shall apply an ordered, seedable stack of additive noise sources (at minimum atom shot noise and a vibration residual) to the clean signal, deterministic given the seed.

**FR-13** (*Atmospheric gravity-gradient noise*). The system shall provide atmospheric GGN (Section 4.14) as a physically-modelled noise source — an infrasound (pressure) channel and a temperature (turbulence) channel, each realised as a stochastic density field through the gravity kernel and differenced across the baseline — with the parameters of Table 1 and the detector depth $z_0$ configurable.

**FR-14** (*State bundle output*). The system shall emit the full per-measurement state bundle (Table 8) with a leading measurement-cadence time axis, a per-detector signal channel, and the time-resolved source tracks. The *complete* record is the default; *field selection* (computing a field only when requested) is a cost/storage optimisation, never a change to what the bundle means or to any imposed task.

**FR-15** (*Track ground truth*). The system shall emit, as ground truth, the time-resolved track of every source (position over time, hence velocity and trajectory) and the target count; for multiple targets this is a set of tracks. A supervised Gradar tracker (Section 5) would use these as its target, but the engine emits them as ground-truth fields and imposes no such role.

**FR-16** (*Signal decomposition and shape descriptors*). The system shall optionally emit the per-channel ground-truth decomposition of `signal` — the phase from target masses, from atmospheric GGN, from ULDM, and the additive measurement noise, which sum to `signal` — and the per-interferometer phases before the double-difference. It shall optionally emit the static per-source shape descriptors (mass, inertia tensor, principal moments and axes, gravitational quadrupole), all derived from `source_cloud`. These are *optional* cost choices, not changes to the bundle's meaning.

**FR-17** (*Single and streaming evaluation*). The system shall provide `run` (one explicit scenario, returning one bundle) and `stream` (an endless prior-sampled iterable yielding batched bundles).

**FR-18** (*Derived outputs*). The system shall provide the Lomb–Scargle periodogram (Section 4.10) as a first-class derived field for non-uniformly sampled series, with the uniform FFT as the fast path for evenly-sampled data.

**FR-19** (*Identifiability analysis*). The system shall, using the differentiable forward model (NFR 8), expose the Fisher-information / Cramér–Rao bound on the recoverable source parameters — the track (position and velocity) covariance given the array geometry and SNR, the quadrupole detectability and shape-class model-selection power (Section 5.4), and a

multi-target resolvability measure (Section 5).

**FR-20** (*Configuration and reproducibility*). The system shall expose a single typed configuration schema to both the SDK (dataclasses) and CLI (TOML/YAML + flag overrides) with no duplicated parameter list, such that (config, seed) fully determines a run.

**FR-21** (*Python SDK*). The system shall provide a Python package (maturin/PyO3 over `sim-core`) returning torch-ready tensors, runnable inside `DataLoader` workers with the GIL released during compute and prefetch enabled.

**FR-22** (*Command-line interface*). The system shall provide a console entry point wrapping `run`/`stream` for producing deliberately frozen bundles (shared eval sets, caches).

**FR-23** (*Viewer*). The system shall provide a native viewer rendering signal and periodogram traces and a 3D scene (baseline, atom clouds, source body on its trajectory), source-agnostic so that a model prediction can be overlaid on ground truth.

**FR-24** (*Optional persistence*). The system shall optionally serialise bundles tensor-natively (e.g. safetensors / WebDataset; Zarr/HDF5 for volumetric fields) for a frozen cache, outside the training loop.

### 6.2 Non-functional requirements

**NFR-1** (*Accuracy*). On the point-mass anchors of Table 3 the system shall reproduce the oracle to $\geq 99\%$; voxelised primitives shall converge to the analytic potentials of Section 4.7 as resolution increases.

**NFR-2** (*Generation throughput*). Live generation shall overlap training without starving the GPU — achieved through multi-process `DataLoader` workers, GIL release during the Rust call, and prefetch. Generation speed, not storage, is the design constraint.

**NFR-3** (*Reproducibility*). (config, root seed) shall fully determine every output tensor. Stochasticity shall use a counter-based / splittable generator with one stream per scenario index derived from the root seed, so a run is independent of `DataLoader` worker count and scheduling — naive per-worker seeding would make the output depend on the parallel layout.

**NFR-4** (*Portability*). A single codebase shall run on Apple Silicon (M3, MPS) and CUDA (RTX 2080 Super, and a secondary CUDA GPU). No datacentre or A100-class assumptions shall appear anywhere.

**NFR-5** (*Modularity*). The four seams of Section 3 shall be the only flex points; deferred features (Appendix A) shall be addable as backends without modifying the gravity core, the forward model, or the state contract.

**NFR-6** (*Testability*). Validation shall rest on committed oracle fixtures with tiered tolerances and localised checks of intermediate quantities, with no Python in CI.

**NFR-7** (*Determinism separation*). The physics shall be deterministic given the seed, with all stochasticity confined to the seeded noise stack and schedule. The CPU reference path shall be bit-reproducible; the GPU hot path, whose reductions reorder, shall be reproducible to the

same tolerance at which it is validated against the reference and the analytic oracle ($\geq 99\%$), which suffices for training data.

**NFR-8** (*Differentiability*). The gravity kernel and the phase integral shall be generic over the scalar type (any type supplying the floating-point operations), so a forward-mode autodiff scalar (dual numbers) can be instantiated without rewriting either — enabling gradient-based inversion and Fisher-information / Cramér–Rao analysis of what the signal can actually identify. This is an M0 decision: free at the outset, a rewrite of both if deferred.

**NFR-9** (*Numerical conditioning*). The gradiometer phase is a difference of differences (two arms, then two interferometers) of individually large potentials, so it is prone to catastrophic cancellation at large source distance. The implementation shall difference potentials *before* summation, or use compensated summation, so a $\geq 99\%$ oracle match cannot be lost to $f64$ cancellation rather than physics.

**NFR-10** (*Deterministic reduction*). The element sum shall reduce via per-thread, cache-line-padded partial accumulators combined in a fixed order, making the result bit-identical regardless of thread or worker count. The same structure removes false sharing — independent threads writing one cache line is measured at several hundred percent overhead — so determinism and speed are one decision, not a trade-off.

**NFR-11** (*Long-run integration stability*). ODE-derived motion (the pendulum's timing and free rotation's orientation) shall use a symplectic integrator (semi-implicit Euler or leapfrog), not explicit Euler or RK4, so energy stays bounded over arbitrarily long runs. For a periodic source whose value is its spectrum, energy drift is spectral drift.

# 7 System Components

Each component lists its responsibility, its interface, and its key algorithms. Formal contracts follow in Section 8.

## 7.1 `gravity` — the gravity kernel

**Responsibility.** Evaluate $V, \mathbf{g}, \Gamma$ for elements, clouds and oracle primitives; reduce a rigid cloud to multipole moments; route near/far. **Features.** Element types (point, softened sphere, Nagy cube); the `Cloud` (`GravitySource`) switching on distance between direct sum and multipole expansion, caching moments to `multipole_order`; analytic primitives as oracle; tensor symmetry/trace invariants. The dependency-light inner module and the most-tested code. **Layout.** The cloud is stored structure-of-arrays (`xs`, `ys`, `zs`, `ms`) in cache-line-aligned buffers, so the near-field sum streams sequentially and vectorises (one reciprocal-distance evaluation per SIMD lane); the far-field multipole data is array-flattened rather than pointer-linked, keeping traversal sequential rather than random. A fat element carrying material or temperature would drag cold bytes through every cache line of the hot loop — the minimal cloud (Section 3.5) is the cache-optimal one. The kernel is generic over the scalar type (NFR 8). **Compute.** The field evaluation runs on the GPU hot path (Section 3.4); a CPU reference path (rayon) is retained for validation and as a Python-free fallback. The kernel is generic over the scalar type (NFR 8). **Non-features (v1).** No adaptive multipole/FMM; the wgpu kernel is f32 (differential-first, NFR 9), f64 only on the CPU/CUDA paths.

## 7.2 `source` — source dynamics and bodies

**Responsibility.** Import meshes to clouds; hold rigid bodies; evolve pose via a `Trajectory`; expose `SourceDynamics`. **Features.** Voxel import (occupancy fill at `voxel_size`, COM and moments at import); `Rigid` (cloud + trajectory, reduced once); the motion library (Section 7.3); a parameterised body library (sphere, cuboid, cylinder, sheet, mesh) so a scenario can name a shape that also exists as an oracle. **Parked behind** `SourceDynamics` (Appendix A): articulated, deforming, fluid.

## 7.3 The motion model

A rigid-body motion factors into four orthogonal parts, with named conveniences wrapping the core so the config never assembles primitives for the common cases.

```
trait Trajectory { fn pose_at(&self, t: f64) -> Pose; }

trait Path   { fn point_at(&self, s: f64) -> Vec3; fn tangent_at(&self, s: f64) -> Vec3; }
//   impls: Line, Arc, Circle, Polyline, Bezier        (curve geometry only)
trait Timing { fn s_at(&self, t: f64) -> f64; }
//   impls: ConstSpeed, Trapezoid, Dwell, OscillateAlong, PendulumODE
enum Orient  { Fixed(Quat), Tangent, Spin { axis: Vec3, rate: f64 },
               Libration { axis: Vec3, amp: f64, freq: f64 },  // prescribed rocking
               FreeRotation { omega0: Vec3 } }                 // Euler top; ODE; inertia
    from cloud
struct Placed { path: Box<dyn Path>, timing: Box<dyn Timing>, orient: Orient, place:
    Isometry3 }

struct Sum(Vec<Box<dyn Trajectory>>);              // superpose (compose poses): oscillation
     on a pass
struct Sequence(Vec<(f64, Box<dyn Trajectory>)>);  // concatenate in time, continuity at
    joins
```

A `Path` is curve geometry only; a `Timing` law sets how arc length advances with time; an `Orient` law sets how the body rotates as it travels; and a rigid `place`ment (an SE(3)) drops the canonical motion at any point in any plane — so "rotated in any plane" costs nothing per motion. Separating `Path` from `Timing` matters physically: the timing shapes the signal as much as the path, and is what turns a line into a lift. Two combinators compose motions: `Sum` superposes (an oscillation carried along a pass), and `Sequence` concatenates in time — so a right-angle turn and a lift are *sequenced* passes, not new primitives (Table 6).

Table 6: Named motions as path × timing over the factored core.

| Convenience | Path | Timing |
|---|---|---|
| `LinearPass` | line | const speed / accel profile |
| `Oscillation` | segment | oscillate |
| `Circular` | closed circle | const angular speed |
| `Lift` | line | accel–cruise–decel–dwell |
| curved pass | arc / bezier | const speed |
| right-angle turn | polyline (+ fillet) | const speed |
| `Pendulum` | arc | ODE-derived (anharmonic at large amplitude) |

**Physical notes.** A sharp turn is a velocity discontinuity — an infinite acceleration and an unphysical signal transient — so a turn defaults to a fillet arc or a decelerate–turn–accelerate; a bezier's natural parameter is not arc length, so uniform speed needs arc-length reparameterisation; and the `Pendulum` is the one motion whose timing is *derived* ($\ddot{\theta} = -(g/L)\sin\theta$) rather than prescribed — at large amplitude it is anharmonic and injects harmonics at $nf_0$

(Section 4.10), while the spherical pendulum is the chaotic Tier-3 case (Table 9), and its ODE is integrated symplectically (NFR 11) so the swing's energy — and thus its spectral peaks — do not drift over a long run. **Rotation.** The `Orient` law turns the body as it travels, and is itself a source class: `Spin` (steady), `Libration` (prescribed rocking), and `FreeRotation` — the torque-free Euler top, the *second* ODE motion after the pendulum, integrated symplectically (NFR 11). Rotation about a fixed centre of mass holds the monopole fixed and the dipole identically zero, so its entire time-varying signal lives in the quadrupole and higher moments: it is the *pure-quadrupole* complement of monopole-dominated translation and a clean probe of the shape rung (Section 5.4), modulating principally at twice the spin rate. It signals only for an *asymmetric* body — a body spun about an axis of symmetry presents an invariant mass distribution, giving a static field and a flat $\Delta\Phi$ (an invariant, Table 9). For `FreeRotation` with three distinct principal moments the intermediate axis is unstable, producing periodic $\sim$180° flips (the tennis-racket / Dzhanibekov effect); but the free top is *integrable, not chaotic* — the flips are a separatrix effect, in deliberate contrast to the genuinely chaotic spherical pendulum (Table 9). Because the inertia tensor and the gravitational quadrupole are the same second moment of the cloud (Figure 4 ff.), the tumble's principal axes *are* the quadrupole's, so a tumbling body over-constrains its own shape — its instantaneous quadrupole fixes the anisotropy, its flip cadence the moment ratios, and the two must agree.

### 7.4 `instrument` — gradiometer array and forward model

**Responsibility.** Hold AION-10 geometry; place the detector array; construct arms; implement the `PhaseModel` and measurement sampling. **Features.** The `Gradiometer` struct (Table 2 defaults); an `Array` of gradiometers at configurable ground `Placements` (Section 4.2), $N$=1 being a single device; arm construction (Section 4.5), shared across detectors; `PropagationIntegral` (Equation (6)) evaluated per detector with configurable quadrature; the `QuasiStaticGradient` alternative; cadence/Nyquist and schedule bookkeeping. The double-difference stays *within* a detector; cross-detector structure is derived downstream, never a new primitive.

### 7.5 `noise` — the noise stack

**Responsibility.** Add realism as an ordered, seedable stack. **Features.** `ShotNoise`$\{n_{\text{atoms}}, C\}$; `VibrationResidual` {psd, rejection}; `AtmosphericGgn` (Section 4.14: an `Infrasound` channel with a closed-form per-shot phase, and a `Temperature` channel sampling the Greenwood–Tarazano spectrum — both realised *through* the gravity kernel as a stochastic density field, then differenced across the baseline); `Vec<Box<dyn NoiseSource>>` applied in order under one seeded RNG; may run empty.

### 7.6 `scenario` and prior

**Responsibility.** Assemble a runnable unit; sample randomised scenarios. **Features.** `Scenario` (sources, array, ULDM signal, noise, schedule, duration, fields); the measurement `Schedule` (Section 4.12: jitter, failures, gaps, Poisson transients); the `Prior` over every parameter (body, mass, distance/closest-approach, speed, frequency, orientation, phase, *target count*, array placement, noise and schedule levels) — the prior is the experiment, and its closest-approach distribution sets `voxel_size` and `far_cutoff`. A dedicated confusion-pair family (instantaneously mass–distance degenerate, Section 5) is part of the prior.

### 7.7 `uldm` — the dark-matter signal

**Responsibility.** Evaluate Equation (14) as an additive common-mode channel. **Features.** Closed-form $\Delta\Phi_\phi(t)$ from Table 4, identical across the array, toggleable per scenario; cheap and deterministic, evaluated outside the gravity kernel.

### 7.8 `config` — the typed schema

**Responsibility.** One typed config in `sim-core`, exposed to both front-ends; home of parameter validation. Resolved config + seed is the reproducibility record. Full schema in Table 7.

### 7.9 `sdk` — Python bindings and CLI

**Responsibility.** Marshal and parameterise the Rust call; no physics in Python. **Features.** `run(...)`→`StateBundle` and `stream(prior,...)`→ `IterableDataset`; `DataLoader`-worker execution with GIL release and prefetch; the Lomb–Scargle periodogram and the track labels as derived/ground-truth fields; `sdk.view(...)`; dataclass config mirror; a CLI console entry point calling the same functions. Targets M3 (MPS) and CUDA; no datacentre assumptions.

### 7.10 `viewer` — debug/inspection

**Responsibility.** Render a state (truth or prediction), interactively. **Features.** `eframe`/`egui` + `egui_plot` (signal/periodogram) + a small `wgpu` 3D viewport (baseline, clouds, source on trajectory, optional field slice); prediction overlay through the same path. Lightweight; not the main mode.

Table 7: Configuration schema (grouped; every field defaulted).

| Group | Keys |
|---|---|
| discretisation | `voxel_size`, `element`∈ {point,sphere,cube}, `multipole_order`, `softening`, `far_cutoff` |
| instrument | `baseline`, `launch_velocity`, `T`, `cycle_time`, `n_kicks`, `species`, `g`, `fine_dt`, `n_clouds`, `phase_model` |
| array | `placements` (per-detector ground position + tower base); $N$=1 default |
| uldm | enable flag + Table 4 parameters ($m_\phi$, $\rho_\phi$, $\theta$, couplings) |
| noise | ordered stack (`shot`$\{n_{\text{atoms}}, C\}$, `vibration`{psd,rejection}, `atmospheric`{infrasound$\langle\delta p, \theta\rangle$, temperature$\langle c_T, \Lambda, U, T_0\rangle$, depth $z_0$}), `seed` |
| body / motion | body: primitive params or mesh path, `density`/target mass; motion: path × timing × orient + placement, or a named motion + params; `n_targets` for multi-target |
| schedule | `duration`; `jitter`, `p_fail`, duty `gaps`, Poisson `transients`{rate, lift params, `excise_pad`} |
| compute | `device`∈ {cpu,wgpu,cuda}, `precision`∈ $\{f32, f64\}$ |
| prior | distributions over all of the above (`stream`) |
| fields | which fields to compute/return + grid resolution for volumetric ones |

## 8 Implementation Contracts

The four seams are specified below with preconditions, postconditions and invariants. A backend that satisfies its contract is a valid drop-in.

**`GravitySource`**

**Methods.** `potential(r)`→`f64`, `field(r)`→`Vec3`, `gradient(r)`→`Mat3`.
**Pre.** `r` finite; for `PointMass`, $\mathbf{r} \neq \mathbf{r}_0$.
**Post.** `field` $= -\nabla$ `potential` and `gradient` $= -\nabla\nabla$ `potential`, each consistent to numerical tolerance; `gradient` symmetric; tr(`gradient`) $= 0$ in vacuum.
**Invariant.** Linearity: for a `Cloud`, `potential` $= \sum_i$ `element`$_i$`.potential`, and far-/near-

field paths agree at the cutoff to tolerance. Purity: no observable state mutation.

**`SourceDynamics`**

**Method.** `cloud_at(t)`→`Cloud` — the discretised $\rho(\mathbf{x}, t)$ (grid cells or particles, each a mass element).
**Pre.** `t` within the scenario window.
**Post.** Total mass conserved: $\sum_i m_i = M$ independent of `t` (for rigid and incompressible sources). `Rigid` satisfies `cloud_at`$(t)$ = `pose_at`$(t)\otimes$`source_cloud` with `pose_at` continuous (and $C^1$ where velocity/accel are requested).
**Invariant.** Determinism: `cloud_at` is a pure function of `t` and fixed parameters.

**`PhaseModel`**

**Method.** `delta_phi(src, instr, t)`→`f64` — $\Delta\Phi_\ell$ in radians.
**Pre.** `src` queryable on $[t - 2T, t]$; `instr` arms built.
**Post.** Returns Equation (7); linear in source mass (`delta_phi`$(\alpha m) = \alpha$ `delta_phi`$(m)$ to tolerance); deterministic. For `QuasiStaticGradient`, agrees with `PropagationIntegral` in the uniform-field limit.

**`NoiseSource`**

**Method.** `add(t, clean, rng)` — additive, in place.
**Pre.** `clean` of length $N$; `rng` seeded.
**Post.** Additive (`out` = `clean`+`noise`); deterministic given the seed; zero-mean for shot/vibration. Order in the stack is significant and preserved.

**`StateBundle` (data contract)**

Leading axis $T$ is the measurement cadence (one entry per sample), carrying the actual timestamps $\{t_\ell\}$ (non-uniform under a schedule). The signal has a per-detector channel axis $D$, and the array geometry is the static `detector_placement`$(D, 7)$. Fields and shapes per Table 8, grouped by what they describe: the source's *motion* (position, orientation, and linear/angular velocity and acceleration — `source_angular_velocity` carries the instantaneous spin axis as its direction, the rate as its magnitude); its static *shape and inertia* (the cloud, plus the derived mass, inertia tensor, principal moments/axes and quadrupole, all reductions of the one second moment $\mathsf{C}$); the *array*; and the *signal* with its optional ground-truth channels (`signal_targets`, `_atmospheric`, `_uldm`, `_noise` sum to `signal`). Primary state is `source_cloud` + the per-tick motion + the noise realisation; the shape descriptors, the channel decomposition, the world cloud and the gravity field are *derived* and computed iff requested. `source_position` (the Gradar target, Section 5) and `n_sources` are ground truth; `mask` flags transient-contaminated cycles.

# 9 Critical Algorithms

Rust-flavoured pseudocode for the load-bearing paths. Types are illustrative; error handling is elided.

## 9.1 The seams

```rust
trait GravitySource {
    fn potential(&self, r: Vec3) -> f64;   // V(r)          [J/kg]
    fn field(&self, r: Vec3) -> Vec3;      // g = -grad V   [m/s^2]
```

```
    fn gradient(&self, r: Vec3) -> Mat3;   // Gamma_ij     [1/s^2]
}

trait SourceDynamics {
    fn cloud_at(&self, t: f64) -> Cloud;   // discretised rho(x,t)
}

trait PhaseModel {
    fn delta_phi(&self, src: &dyn SourceDynamics,
                 instr: &Instrument, t_meas: f64) -> f64;   // DeltaPhi_l [rad]
}

trait NoiseSource {
    fn add(&self, t: &[f64], clean: &mut [f64], rng: &mut StdRng);
}

struct Composite(Vec<Box<dyn SourceDynamics>>);   // clouds superpose
```

## 9.2 Arm trajectory construction

Built once per instrument; the same flight repeats each measurement cycle.

```
fn build_arms(instr: &Instrument) -> [Arm; 4] {
    let v_rec = instr.n_kicks as f64 * HBAR * instr.k() / instr.m_atom;  // eq (recoil)
    let dt = instr.fine_dt;
    let mut arms = Vec::new();
    for z0 in [0.0, instr.baseline] {            // lower IFO, upper IFO
        // lower arm: launch at u0, +v_rec kick at T
        arms.push(integrate_arm(z0, instr.u0,         +v_rec, instr.T, instr.g, dt));
        // upper arm: +v_rec at launch, -v_rec kick at T
        arms.push(integrate_arm(z0, instr.u0 + v_rec, -v_rec, instr.T, instr.g, dt));
    }
    [arms[0], arms[1], arms[2], arms[3]]
}

fn integrate_arm(z0: f64, v0: f64, kick_at_T: f64,
                 T: f64, g: f64, dt: f64) -> Arm {
    let mut z = z0; let mut v = v0; let mut path = Vec::new();
    let n = (2.0 * T / dt) as usize;
    for i in 0..=n {
        let t = i as f64 * dt;
        path.push((t, z));
        if (t - T).abs() < 0.5 * dt { v += kick_at_T; }   // pi-pulse momentum swap
        z += v * dt - 0.5 * g * dt * dt;                  // free fall under -g
        v -= g * dt;
        if z <= z0 && v < 0.0 { z = z0; v = 0.0; }         // reflecting floor (landed)
    }
    Arm::from_samples(path)   // interpolable over [0, 2T]
}
```

## 9.3 The propagation-phase integral (forward model)

The core of the simulator: Equations (6) and (7).

```
impl PhaseModel for PropagationIntegral {
    fn delta_phi(&self, src: &dyn SourceDynamics,
                 instr: &Instrument, t_meas: f64) -> f64 {
        let arms = instr.arms();                  // [lower1, upper1, lower2, upper2]
        let prefactor = instr.m_atom / HBAR;
        // per-interferometer phase: integral of V(upper) - V(lower) over the flight
```

```
        let dphi = |lower: &Arm, upper: &Arm| -> f64 {
            integrate(t_meas - 2.0 * instr.T, t_meas, instr.fine_dt, |t| {
                let tf = t - (t_meas - 2.0 * instr.T);      // local flight time in [0,2T]
                let cloud = src.cloud_at(t);                // source has moved to time t
                let ru = vec3(0.0, 0.0, upper.z_at(tf));    // atoms move only in z
                let rl = vec3(0.0, 0.0, lower.z_at(tf));
                cloud.potential(ru) - cloud.potential(rl)   // eq (cloud potential)
            })
        };
        let dphi_1 = prefactor * dphi(&arms[0], &arms[1]);  // lower IFO (z0 = 0)
        let dphi_2 = prefactor * dphi(&arms[2], &arms[3]);  // upper IFO (z0 = baseline)
        dphi_2 - dphi_1                                      // gradiometer double
    difference
    }
}
```

## 9.4 Cloud field with near/far switch

```
impl GravitySource for Cloud {
    fn potential(&self, r: Vec3) -> f64 {
        if (r - self.com).norm() > self.far_cutoff {
            self.multipole.potential(r)            // far field: moments
        } else {
            -G * self.elements.iter()              // near field: direct sum, eq (cloud pot)
                .map(|e| e.mass / (r - e.pos).norm())  // element kind sets the kernel
                .sum::<f64>()
        }
    }
    // field(), gradient() analogous (multipole or summed element derivatives)
}
```

## 9.5 Multipole reduction (once per body)

```
fn reduce(elements: &[Element]) -> Multipole {
    let m: f64 = elements.iter().map(|e| e.mass).sum();
    let com: Vec3 = elements.iter().map(|e| e.mass * e.pos).sum::<Vec3>() / m;
    let mut q = Mat3::zeros();                     // quadrupole about COM (dipole = 0 there)

    for e in elements {
        let d = e.pos - com;
        q += e.mass * (3.0 * d.outer(&d) - d.norm_squared() * Mat3::identity());
    }
    Multipole { m, com, q }                        // re-expressed from pose each tick (
    rotate)
}
```

## 9.6 Mesh voxelisation

```
fn voxelise(mesh: &Mesh, voxel_size: f64, density: f64, kind: Element) -> Cloud {
    let bbox = mesh.bounding_box();
    let mut elements = Vec::new();
    for cell in bbox.grid(voxel_size) {            // occupancy fill
        if mesh.contains(cell.center()) {
            let m = density * voxel_size.powi(3);
            elements.push(Element::new(kind, cell.center(), m));
        }
    }
```

```
    let multipole = reduce(&elements);
    Cloud { elements, multipole, com: multipole.com, far_cutoff: /* from config */ }
}
```

### 9.7 Generation: `run` and `stream`

On CPU this is the reference loop below; the batch path dispatches the same evaluation as the GPU outer product (scenario × detector × timestep × field point, Section 3.4).

```
fn run(scenario: &Scenario, fields: FieldSet) -> StateBundle {
    let array = &scenario.array;                    // detectors at placements
    let t_meas = scenario.schedule.sample();        // measurement times (jitter/gaps/fails)
    let mut bundle = StateBundle::with_capacity(t_meas.len(), array.len(), fields);
    for (l, &t) in t_meas.iter().enumerate() {      // measurement-cadence axis
        for (d, det) in array.iter().enumerate() { // per detector
            let mut phi = scenario.phase_model.delta_phi(&scenario.sources, det, t);
            if scenario.uldm.enabled { phi += scenario.uldm.phase(t); }  // common-mode
            bundle.signal[(l, d)] = phi;
        }
        if fields.tracks { bundle.record_tracks(l, &scenario.sources, t); }  // label
        if fields.source { bundle.record_source(l, &scenario.sources, t); }
        if fields.field  { bundle.record_field(l, &scenario.sources, array, t); }
    }
    bundle.mask = scenario.schedule.contamination(&t_meas);   // transient tagging
    for noise in &scenario.noise_stack {                      // seeded, ordered
        noise.add(&bundle.t, &mut bundle.signal, &mut bundle.rng);
    }
    if fields.periodogram { bundle.periodogram = lomb_scargle(&bundle.t, &bundle.signal); }
    bundle
}

// stream: endless prior-sampled scenarios, evaluated in parallel DataLoader workers
fn stream(prior: &Prior, fields: FieldSet) -> impl Iterator<Item = StateBundle> {
    std::iter::from_fn(move || {
        let scenario = prior.sample();              // body, motion, distance, freq, noise,
    ...
        Some(run(&scenario, fields))                // GIL released across this call
    })
}
```

## 10 Test Specification

The reference code of [1] is the *golden master*. A sweep of (scenario → outputs) is frozen as committed fixtures — no Python in CI — and asserted against with *tiered* tolerance. Validation is always against the *full*-simulation outputs, never the report's perturbative closed forms (the full simulation supersedes those). Fixtures capture the oracle's own numerical knobs (`fine_dt`, quadrature tolerances, grid sizes) so like is compared with like.

### 10.1 Tiers

Each tier names an explicit error metric; "≥ 99%" otherwise means nothing against a time series.

**Tier 1 — point-mass forward model (tight).**
Identical mathematics on the same trajectories; a 1% gap is a bug, not tolerance. Anchors: the 10 kg vertical pass (≈ 7 mrad) and the 1 kg oscillator (≈ 2 mrad). Metric: relative

error of the series, $1 - \|\text{sim} - \text{oracle}\|_2 / \|\text{oracle}\|_2 \geq 0.99$, and separately $\max|\Delta\Phi|$ within 1%.

**Tier 2 — voxel convergence (the key assertion).**
A voxelised primitive converges to the analytic oracle as `voxel_size` $\to 0$: the steel column ($R = 0.0338\,\text{m}$) against the cylinder integral of Section 4.7 at sampled field points, and the resulting $\Delta\Phi$ to $\geq 99\%$ by the same relative-$L_2$ metric, with the per-point potential error decreasing monotonically in resolution. Convergence *is* the proof the voxel approach is correct.

**Tier 3 — ODE motion (feature-level).**
The chaotic 3D pendulum cannot match pointwise at long horizons, so the metric is spectral: periodogram peak positions within one frequency bin and peak-height ratios within a stated tolerance. Pointwise agreement is explicitly not required.

**Tier 4 — systems and identifiability (end-to-end).**
Two whole-pipeline checks against the reference code: (i) the cylinder-vs-point-mass sweep over radius, height and mass height reproduces the reference's departure-from-monopole curve — which is simultaneously the validation of the multipole-truncation and `far_cutoff` choices, since it is exactly where extended-body shape becomes resolvable; and (ii) the end-to-end ULDM run — coherent signal + GGN oscillators + Poisson lifts on a gapped, jittered schedule — recovers the $f_\phi$ line above the noise floor after excision, Lomb–Scargle and the harmonic notch (Section 4.10). The Gradar track CRB and multi-target resolvability (Section 5) are checked as numerical-floor sanity, not against the oracle.

### 10.2 Localised and invariant checks

Where the oracle exposes intermediates (potential at a point, per-interferometer $\delta\phi$ before differencing), assert on those so a failure localises to a module. Free invariants: $\Gamma$ symmetric and trace-free in vacuum; $\Delta\Phi$ linear in source mass; a far-field cloud reproduces its own monopole.

## 11 Build Milestones

**M0 — gravity kernel + oracle.**
Voxel cloud, element types, multipole evaluator, analytic primitives, generic over the scalar type (NFR 8). *Exit:* T2.1/T2.2 convergence and the T2.4 sweep; INV.1 holds. No dependency on the oracle's runtime — buildable immediately.

**M1 — instrument + forward model + first signal.**
Arm construction + propagation integral on a single detector; Tier-1 fixtures tight, Tier-2 convergence. *Exit:* T1.1 asserts the 7 mrad pass; INV.2 holds.

**M2 — array + ULDM + scenario + schedule + bundle.**
Generalise to the detector array; add the closed-form ULDM channel and the measurement schedule; assemble the bundle (Table 8) with the track labels; field selection works. *Exit:* INV.4 (common-mode ULDM); T4.1 line recovery; multi-target generation.

**M3 — compute + SDK (+ CLI).**
The wgpu/CUDA field-evaluation path (Section 3.4) cross-validated against the CPU reference; `run`/`stream` yield torch tensors; a `DataLoader` with prefetch overlaps a dummy training step (GIL released); Lomb–Scargle output.

**M4 — viewer.**
`egui`/`wgpu`: signal/periodogram plot + 3D scene (array, clouds, sources on trajectories).

**M5 — identifiability.**
The Fisher / Cramér–Rao analysis (Section 5): track covariance, quadrupole detectability and shape-class model selection (Section 5.4), and multi-target resolvability via the differentiable forward model.

**M6+ (parked).**
The learned Gradar tracker; noise-realism depth; non-rigid dynamics (Appendix A).

## 12 Open Decisions and Risks

1. **Inner-integral quadrature.** Mirror the oracle (interpolate arms + $V$, adaptive quadrature) for maximum fidelity, or integrate directly with fixed-step Simpson on `fine_dt`? Affects how cleanly Tier-1 reaches $\geq 99\%$ and generation speed. *Lean:* direct fixed-step, validate against the oracle, fall back to adaptive only if needed.
2. **Extended-body $V$ in generation.** Always direct element sum, or adopt a precomputed-grid + spline for moving extended bodies? *Lean:* direct sum in v1; add a cached-grid path if profiling demands.
3. **Model input representation.** The Gradar tracker's input is the joint multi-detector $\Delta\Phi(t)$; the Lomb–Scargle periodogram is load-bearing for the ULDM channel. Open: whether the tracker consumes raw multi-channel time series, the per-channel spectra, or both. Decides which derived fields the SDK emits as first-class. Does not block the engine.
4. **Per-measurement pose convention.** Report the source pose at $t_\ell$ or at the flight midpoint $t_\ell - T$? Pick one and document it. The same choice fixes the track-label timestamp.
5. **Config DSL.** Flat enums vs a small composable DSL for `Composite` trajectories (needed for the scaffold's many members and multi-target scenes). *Lean:* flat for v1.
6. **Array geometry as a design knob.** The placement that maximises localisation is itself an optimisation against the CRB (Section 5). Open: a fixed default array vs an optimised-per-prior layout. Does not block v1 (default array).
7. **Multi-target association.** Whether to label tracks as an ordered set with a matching loss (Hungarian/JPDA-style) or as an unordered set; affects the loss, not the engine, but the prior must generate consistent ground truth.
8. **Excise vs. mask.** Hand the tracker pre-excised clean data, or contaminated data plus the `mask` so it learns transient robustness. The bundle carries both, so this is a training choice deferred to the consumer.

**Principal risk.** The solver-driven sources of Appendix A lose the oracle entirely (the report uses rigid coherent motion). The mitigation is sequencing: land the rigid forward model on a tight $\geq 99\%$ match *first*, so that when later, harder sources behave oddly the kernel beneath is known-trustworthy.

## A Parked Source Dynamics: the Extension Sketch

Everything past `Rigid` — articulated bodies, fluids (compressible & incompressible), structural vibration, airflow/convection — is addable without touching the gravity core, the forward model, or the state contract, because all reduce to the same currency $\rho(\mathbf{x}, t)$. What differs is only *how* each produces it, which splits every extension into two families.

## A.1 Prescribed vs solved

**Prescribed (Category A).** $\rho(\mathbf{x}, t)$ is a known function of parameters one *evaluates* (a rigid pose, gait kinematics, analytic sloshing modes, a parameterised plume, modal vibration with precomputed modes). Cheap, stateless per tick, parallel; fits live generation.

**Solved (Category B).** $\rho(\mathbf{x}, t)$ is the output of integrating a PDE/particle system forward in time (free-surface CFD, elastodynamics, convection). Stateful, solver-sized, too slow for the training loop (and on M3 / RTX 2080, not close), so it integrates via cache-and-replay; and it loses the oracle (Appendix A.4).

## A.2 The composable seam

The general `SourceDynamics` method is the whole cloud at $t$; `Rigid`'s {`cloud_body, pose_at`} is a specialisation. Three additions make extensions compose:

```
trait SourceDynamics { fn cloud_at(&self, t: f64) -> Cloud; }  // cells or particles
struct Composite(Vec<Box<dyn SourceDynamics>>);                // sum the clouds
trait Driver { fn forcing(&self, t: f64) -> Forcing; }         // one-way coupling
struct CachedField { /* stored rho(x,t) */ }                   // solver plug-point
```

`Composite` is the general "several independent moving things" — and is what the scaffold needs in v1, so it earns its place early; `CachedField` is where every Category-B backend plugs in. The gravity core still only ever sees a cloud.

*Superposition is not coupling.* Combining sources *gravitationally* is trivial: their clouds simply add. But the *physical* interaction between them lives inside the solvers, not in the sum, and is the hard part. A footfall driving a floor is one-way and linear (a point impulse ringing precomputed modes, via the `Driver` hook). A person stirring the air is moving-boundary fluid dynamics — the body is a moving obstacle the air solver must respond to — genuine two-way coupling, firmly in the deep end. "Just different fields interacting" understates this: their gravity adds for free; their physics driving each other is the eventual hard work.

## A.3 Per-phenomenon fidelity ladders

**Articulated bodies (walking human).** The natural first extension and a named GGN source (people, modelled as point masses in [2]). It is *consuming a rigged asset, not building an animation engine*: import a rigged glTF (skeleton, skin weights, clips), sample a clip at time $t$ (interpolating the keyframes), and read off the bone transforms. The engine work — rigging, authoring animations, inverse kinematics, blending, rendering — happens upstream in tools such as Blender or Mixamo and arrives baked in, so the project needs only glTF import, clip sampling, linear-blend skinning and voxelisation; pre-made walking clips (Mixamo, the CMU mocap database, any BVH/glTF humanoid) are a direct shortcut. No new gravity physics is required: the body is a small set of rigid segments (anthropometric ellipsoids/cylinders, masses and lengths from height + total mass) whose relative poses change per tick. Because skinning is almost piecewise-rigid (each vertex follows its dominant bone, blending only across a few centimetres at the joints) and gravity at metre-scale distances cannot resolve that blending, the efficient path is to **voxelise each bone once in bone-local space and apply its transform per frame** — a rigid transform of a fixed cloud, with the per-segment multipole reduction still valid — rather than re-voxelising the deformed mesh every frame (which is the brute-force `Deforming` path, reserved for genuinely deforming bodies a walking human does not need). The motion factorises: the `Trajectory` seam walks the *root*; a `Gait` (joint angles → forward kinematics) animates the limbs. This is *kinematics, not dynamics*: gravity needs only where the mass is, so plausible joint trajectories (parametric or mocap) suffice.

**Fluids — incompressible.** The signal comes from a *moving interface*, not the bulk: a fully-filled rigid container of uniform incompressible fluid is gravitationally static. *Prescribed:* small-amplitude sloshing modes (closed-form for rectangular/cylindrical tanks). *Solved:* free-surface CFD (SPH or level-set/VOF) for large amplitude; pressure projection enforces $\nabla\cdot\mathbf{v} = 0$.

**Fluids — compressible.** Density itself is dynamical (continuity $\partial_t\rho + \nabla\cdot(\rho\mathbf{v}) = 0$ plus an equation of state), so the bulk field moves mass directly — the air case. Prescribed: advecting density anomalies. Solved: compressible/low-Mach Navier–Stokes.

**Structural vibration (footfall → floor).** The report models walls/scaffold as *rigid coherent* oscillation and flags it as an overestimate; the real field is spatially varying. The refinement is *modal*: eigenmodes $\varphi_n(\mathbf{x})$ at frequencies $\omega_n$, displacement $\mathbf{u}(\mathbf{x},t) = \sum_n a_n(t)\varphi_n(\mathbf{x})$, mass redistribution

$$\delta\rho = -\nabla\cdot(\rho_0\,\mathbf{u}) \tag{19}$$

linear in the modal amplitudes. Modes are precomputed once (FE eigenproblem) — or analytic for a simple rectangular floor-plate, $\varphi_{mn} = \sin(m\pi x/L_x)\sin(n\pi y/L_y)$, no solver. A footfall is a point impulse that rings the modes; a walking person (the articulated source) is a `Driver` emitting footfall forces, and *person + floor* is a `Composite` of two sources with one-way coupling, both contributing mass. Ladder: F0 rigid coherent (already supported); F1 modal; F2 full elastodynamics (solver).

**Airflow / convection.** Air-density perturbations move mass; weak ($\rho_{\text{air}} \approx 1.2\,\text{kg/m}^3$) but a named source (atmospheric GGN). The hardest family. *Prescribed:* a buoyant thermal plume (advecting Gaussian); HVAC as a prescribed field. *Solved:* Boussinesq convection (Rayleigh–Bénard) — a genuine CFD sub-project, turbulent and chaotic, so the signal is a weak broadband stochastic field, exactly the regime where notch filters fail and learned regression is needed. The strongest motivation for the ML, and the deep end.

### A.4 Validation reality

Prescribed sources inherit correctness from the validated kernel (a fluid/vibrating cloud is still a cloud), and analytic cases (sloshing/plate modes) are checkable against closed forms. *Solved sources lose the oracle entirely* — validation moves to the solver's own benchmarks (lid-driven cavity, Rayleigh–Bénard, analytic beam/plate response), a separate track. This is why solvers are deep-parked.

### A.5 What to put in v1 now

Four small seams, none the phenomenon itself: (1) `Composite` sources — needed for the scaffold anyway; (2) a cloud that is grid-cells *or* particles, so no solver backend is blocked by representation; (3) `CachedField` reserved as the solver plug-point; (4) a `Driver` hook for footfall→modes coupling.

## B Nomenclature

| Symbol | Meaning |
|---|---|
| $\Delta\Phi_\ell$ | gradiometer differential phase at measurement $\ell$ (the signal) |
| $\delta\phi_\ell$ | single-interferometer phase |
| $V, \Phi$ | gravitational potential per unit mass [J/kg] |
| $\mathbf{g}$ | gravitational acceleration, $-\nabla\Phi$ |
| $\Gamma_{ij}$ | gravity-gradient tensor, $\partial g_i/\partial x_j$ |

| | |
|---|---|
| $\rho(\mathbf{x}, t)$ | mass-density field (the universal source currency) |
| $m_i, \mathbf{r}_i$ | mass and world position of cloud element $i$ |
| $z_u, z_l$ | upper/lower arm trajectories of an interferometer |
| $T$ | pulse separation; flight time $2T$ |
| $\Delta t$ | measurement cadence (sets $f_{\mathrm{N}} = 1/2\Delta t$) |
| `fine_dt` | within-flight integration step |
| $\Delta r$ | gradiometer baseline (interferometer separation) |
| $v_{\mathrm{rec}}$ | LMT recoil/arm-separation velocity, $n_{\mathrm{kick}}\hbar k/m_A$ |
| $k$ | laser wavenumber $2\pi/\lambda$ |
| $M, \mathbf{p}, \mathsf{Q}$ | multipole moments: monopole, dipole, quadrupole |
| $\mathsf{C}, \mathsf{I}$ | second moment $\sum m\,\mathbf{r}\mathbf{r}^{\top}$ and inertia tensor; share axes with $\mathsf{Q}$ |
| $a, r$ | source size and field-point distance; $(a/r)^2$ is the shape-SNR budget |
| $S_k, f_k$ | periodogram value and frequency bin |
| $f_\phi, \omega_\phi$ | ULDM oscillation frequency $m_\phi c^2/h$ and $2\pi f_\phi$ |
| $m_\phi, \rho_\phi$ | ULDM scalar mass and local dark-matter density |
| $\delta\omega_A$ | atomic-transition modulation amplitude (Equation (14)) |
| $\delta\rho, \delta p, \delta T$ | atmospheric density, pressure, temperature fluctuations |
| $\Lambda, U, c_T$ | turbulence outer scale, wind speed, temperature structure constant |
| $T_\phi$ | interferometer-sequence transfer function (noise → phase) |
| $D$ (axis) | number of detectors in the array; $S$ number of sources |
| $\varphi_n, \omega_n, a_n$ | structural eigenmode, frequency, modal amplitude |
| $\mathbf{u}$ | structural displacement field |
| $C, N_{\mathrm{atoms}}$ | fringe contrast, atom number (shot noise) |

# References


[1] G. Parish, *Charting New Frontiers in Dark Matter Detection using Atom Interferometry*, MPhil-to-PhD upgrade report, King's College London / Rutherford Appleton Laboratory, 2026.

[2] J. Carlton and C. McCabe, *Mitigating anthropogenic and synanthropic noise in atom interferometer searches for ultralight dark matter*, Phys. Rev. D **108**, 123004 (2023).

[3] J. Carlton, V. Gibson, T. Kovachy, C. McCabe and J. Mitchell, *Clear skies ahead: characterizing atmospheric gravity gradient noise for vertical atom interferometers*, Phys. Rev. D **111**, 082003 (2025).

[4] L. Badurina *et al.*, *AION: an Atom Interferometer Observatory and Network*, JCAP **05**, 011 (2020).

[5] G. Rosi *et al.*, *Precision measurement of the Newtonian gravitational constant using cold atoms*, Nature **510**, 518 (2014).

[6] J. B. Fixler *et al.*, *Atom interferometer measurement of the Newtonian constant of gravity*, Science **315**, 74 (2007).

[7] D. Nagy, G. Papp and J. Benedek, *The gravitational potential and its derivatives for the prism*, Journal of Geodesy **74**, 552 (2000).

[8] C. Overstreet *et al.*, *Physically significant phase shifts in matter-wave interferometry*, Am. J. Phys. **89**, 324 (2021).

[9] J. D. Scargle, *Studies in astronomical time series analysis. II. Statistical aspects of spectral analysis of unevenly spaced data*, Astrophys. J. **263**, 835 (1982).

[10] A. Arvanitaki, J. Huang and K. Van Tilburg, *Searching for dilaton dark matter with atomic clocks*, Phys. Rev. D **91**, 015015 (2015).

Table 8: State bundle fields (leading time axis $T$ = measurement cadence).

| Field | Shape | Notes |
|---|---|---|
| *Time, and the source's motion (per source $S$, over $T$)* | | |
| `time` | $(T,)$ | timestamps [s] (non-uniform under a schedule) |
| `source_position` | $(S,T,3)$ | COM world position — the localisation target |
| `source_orientation` | $(S,T,4)$ | orientation quaternion wxyz (world ← body) |
| `source_velocity` | $(S,T,3)$ | COM linear velocity |
| `source_angular_velocity` | $(S,T,3)$ | angular velocity $\boldsymbol{\omega}$: direction is the instantaneous spin axis, magnitude the rate |
| `source_accel` | $(S,T,3)$ | COM linear acceleration |
| `source_angular_accel` | $(S,T,3)$ | angular acceleration |
| *The source's shape & inertia (per source $S$, static — one second moment $\mathsf{C}$)* | | |
| `source_cloud` | $(S,N,4)$ | body-frame mass elements $(x,y,z,m)$; the discretised density (fixed) |
| `source_mass` | $(S,)$ | optional: total mass $M$ |
| `source_inertia` | $(S,3,3)$ | optional: inertia tensor $\mathsf{I}$ (body frame) |
| `source_moments` | $(S,3)$ | optional: principal moments $(I_1,I_2,I_3)$ |
| `source_axes` | $(S,3,3)$ | optional: principal axes (body frame; shared by $\mathsf{I}$ and $\mathsf{Q}$) |
| `source_quadrupole` | $(S,3,3)$ | optional: gravitational quadrupole $\mathsf{Q}$ (body frame) |
| *The array (per detector $D$, static)* | | |
| `detector_placement` | $(D,7)$ | per detector: position xyz + orientation quat; static (v1 vertical) |
| *The signal (per measurement $T$, detector $D$) and its ground-truth channels* | | |
| `signal` | $(T,D)$ | gradiometer $\Delta\Phi$ (targets + atmospheric + ULDM + noise) [rad] |
| `mask` | $(T,)$ | transient-contaminated cycles (excise/mask) |
| `signal_targets` | $(T,D)$ | optional: phase from the target masses alone (clean) |
| `signal_atmospheric` | $(T,D)$ | optional: phase from atmospheric GGN |
| `signal_uldm` | $(T,)$ | optional: ULDM common-mode phase (identical across detectors) |
| `signal_noise` | $(T,D)$ | optional: additive measurement noise (shot, vibration) |
| `signal_per_ifo` | $(T,D,2)$ | optional: the two single-interferometer phases (pre double-difference) |
| *Field samples and derived spectra (optional, heavier)* | | |
| `field_at_clouds` | $(T,D,2,3)$ | optional: $\mathbf{g}$ at each atom cloud ($n_c$=2) |
| `gradient_at_clouds` | $(T,D,2,3,3)$ | optional: gradient tensor $\Gamma$ at each cloud |
| `field_grid` | $(T,X,Y,Z,3)$ | optional: $\mathbf{g}$ on a grid (storage-dominant) |
| `periodogram` | $(F,D)$ | optional derived: Lomb–Scargle PSD of `signal` |
| *Counts & metadata* | | |
| `n_sources` | — | number of sources $S$ |
| `meta` | — | resolved config (incl. each source's motion type & parameters) + seed |

Table 9: Test matrix (representative).

| ID | Tier | Scenario | Assertion | Tol. |
|---|---|---|---|---|
| T1.1 | 1 | 10 kg vertical pass, $D = 10\,\mathrm{m}$ | $\Delta\Phi(t)$ vs oracle; peak $\approx 7\,\mathrm{mrad}$ | 1% |
| T1.2 | 1 | 1 kg oscillator, $D_0 = 5\,\mathrm{m}$ | $\Delta\Phi(t)$, peak $\approx 2\,\mathrm{mrad}$ | 1% |
| T2.1 | 2 | voxel sphere vs analytic sphere | $V(\mathbf{r})$ at sampled points, vs resolution | conv. |
| T2.2 | 2 | voxel cylinder (steel column) vs oracle integral | $V(\mathbf{r})$ and $\Delta\Phi$ | $\geq 99\%$ |
| T2.3 | 2 | voxel cuboid (concrete wall, $0.225 \times 6 \times 12\,\mathrm{m}$) vs oracle | peak $\approx 50\,\mu\mathrm{rad}$ | $\geq 99\%$ |
| T2.4 | 2 | cylinder vs point mass, swept $R, H, h_0$ | departure-from-monopole curve; multipole/cutoff | match |
| T3.1 | 3 | 3D pendulum | periodogram peak positions + ratios | feat. |
| T4.1 | 4 | ULDM + GGN + lifts, gapped/jittered | $f_\phi$ line recovered post excision/notch | spec. |
| T4.2 | 4 | single target, array | track CRB; position from joint signal | floor |
| T4.3 | 4 | multi-target (degenerate pair) | resolvability; tracks diverge over time | floor |
| T4.4 | 4 | free-rotation tumbler (intermediate axis) | orientation tracked through flips; separatrix pair | floor |
| INV.1 | — | arbitrary cloud in vacuum | $\Gamma = \Gamma^\top$, $\mathrm{tr}\,\Gamma = 0$ | $\epsilon$ |
| INV.2 | — | scale source mass by $\alpha$ | $\Delta\Phi \to \alpha\Delta\Phi$ | $\epsilon$ |
| INV.3 | — | distant cloud | cloud field = monopole field | tol. |
| INV.4 | — | ULDM only, array | signal identical across detectors (common-mode) | $\epsilon$ |
| INV.5 | — | symmetric body spun about its symmetry axis | $\Delta\Phi$ flat (static field) | $\epsilon$ |